

Major Project Report on
**STOCKAI: LLM-BASED MULTI-AGENT STOCK MARKET
SIMULATION**

A report submitted in partial fulfilment of the requirements for the Award of Degree of

BACHELOR OF ENGINEERING

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

By

MOHAMMED SAMIULLAH - 1604-22-747-073

MOHAMMED RIYAN OZAIR - 1604-22-747-079

MOHAMMED NABEEL RIZWAN - 1604-22-747-107

Under the Guidance of

Mrs. Bushra Siddiqua



**DEPARTMENT OF COMPUTER SCIENCE AND ARTIFICIAL
INTELLIGENCE**

**MUFFAKHAM JAH COLLEGE OF ENGINEERING AND
TECHNOLOGY**

(Affiliated to Osmania University)

Mount Pleasant, 8-2-249, Road No. 3, Banjara Hills, Hyderabad 500034, Telangana, State, India

YEAR OF SUBMISSION: 2025– 2026



MUFFAKHAM JAH COLLEGE OF ENGINEERING AND TECHNOLOGY

Mount Pleasant, 8-2-249 to 267, Road No. 3, Banjara Hills, Hyderabad - 500 034, Telangana, State, India

CERTIFICATE

This is to certify that the **Major Project Report** on “**StockAI: LLM-Based Multi-Agent Stock Market Simulation**” submitted by **Mohammed Samiullah (1604-22-747-073)**, **Mohammed Riyan Ozair (1604-22-747-079)**, **Mohammed Nabeel Rizwan (1604-22-747-107)** is work done by them under our guidance and submitted during the **2025-2026** academic year, in partial fulfilment of the requirement for the Award of the Degree, **Bachelor of Engineering in Artificial Intelligence and Data Science**. This work is not submitted elsewhere for a degree.

Mrs. Bushra Siddiqua
Assistant Professor,
Internal Guide,
CS&AI Dept, MJCET

Prof. Uma N. Dulhare
Head, CS&AI Dept.,
MJCET

External Examiner

DECLARATION

This is to certify that the work reported in the Major Project Report entitled **StockAI: LLM-Based Multi-Agent Stock Market Simulation** is a record of work done by **Mohammed Samiullah (1604-22-747-073), Mohammed Riyan Ozair (1604-22-747-079), and Mohammed Nabeel Rizwan (1604-22-747-107)** in the **Department of Computer Science and Artificial Intelligence, Muffakham Jah College of Engineering and Technology, Osmania University**, the report is based on the Major Project work done entirely by our team and not copied from any other source nor submitted elsewhere for a degree.

Mohammed Samiullah - 1604-22-747-073

Mohammed Riyan Ozair - 1604-22-747-079

Mohammed Nabeel Rizwan - 1604-22-747-107

ACKNOWLEDGEMENT

It is indeed with a great sense of pleasure and immense sense of gratitude that I acknowledge the help of the following individuals.

Firstly, I would like to thank my Head of the Department, **Prof. Uma N. Dulhare**, for her constructive criticism and continuous encouragement throughout my Major Project.

I would like to express my sincere gratitude to my Project Guide, **Mrs. Bushra Siddiqua, Assistant Professor**, Department of Computer Science and Artificial Intelligence, for her valuable guidance, support, and motivation in accomplishing the Major Project Phase 1 objectives.

I would also like to thank my Project Coordinator, **Mr. Mohammed Abdul Khader**, Assistant Professor, Department of Computer Science and Artificial Intelligence, for his constant support, coordination, and timely suggestions throughout the project work.

Finally, I would like to take this opportunity to thank my family for their unwavering support. I sincerely acknowledge and thank all those who have directly or indirectly supported me in the completion of this Major Project.

Mohammed Samiullah - 1604-22-747-073

Mohammed Riyan Ozair - 1604-22-747-079

Mohammed Nabeel Rizwan - 1604-22-747-107

TABLE OF CONTENTS

CONTENT	PAGE NOS
I. ABSTRACT	i
II. LIST OF FIGURES	ii
III. LIST OF TABLES	iii
IV. ACRONYMS	iv
CHAPTERS	
1. INTRODUCTION	1-3
1.1 Introduction	1
1.2 Aim & Objectives	1
1.3 Reason for Project	2
1.4 Problem Statement	2
1.5 Scope	3
1.6 Summary	3
2. LITERATURE SURVEY	4-25
2.1 Introduction	4-5
2.2 Classical Time-Series Forecasting	5-7
2.3 Stylized Facts and Financial Data Complexity	7-8
2.4 Machine Learning for Stock Prediction	8-9
2.5 Deep Learning and Sequence Models	9-10
2.6 Sentiment, News and Event-Driven Market Models	10
2.7 Reinforcement Learning in Trading	11
2.8 Algorithmic Trading and Strategy Evaluation	12
2.9 Agent-Based Computational Finance	12-13
2.10 Market Microstructure and Limit Order ideas	13-14
2.11 Behavioural Finance and Trading Psychology	14-15
2.12 Large Language Models and Generative Agents	15-16
2.13 StockAgent and LLM-Based Multi-Agent Trading Simulation	16-17
2.14 Relevance of StockAgent to StockAI v2.0	17-18
2.15 Research Platforms and Experiment Reproducibility	18-19
2.16 Explainable AI in Financial Decision Support	19-20
2.17 Live-Market Intelligence and Synthetic-to-Real Comparision	20
2.18 Full-Stack Architecture for Financial AI Tools	21
2.19 Comparison of Existing Approaches	21-22
2.20 Identified Literature Gaps	23
2.21 Relevance to StockAI v2.0 Modules	23-24
2.22 Summary and Conclusion	24-25
3. EXISTING SYSTEM	26-27
3.1 Introduction	26
3.2 Limitations	26-27
3.3 Problem Statement	27
4. PROPOSED SYSTEM	28-32
4.1 Introduction	28

4.2 Advantages	29
4.3 Specifications of the Proposed System	29-32
4.3.1 Proposed Architecture	30-32
5. SYSTEM ANALYSIS	33-38
5.1 Introduction	33
5.2 Feasibility Study	33-36
5.2.1 Technical Feasibility	33
5.2.2 Operational Feasibility	34
5.2.3 Economical Feasibility	34-36
5.2.4 Legal Feasibility	36
5.3 System Implementation	36
5.4 Functional Requirements	37
5.5 Non-Functional Requirements	37-38
5.6 Hardware & Software Requirements	38
6. SYSTEM DESIGN	39-45
6.1 System Architecture Design	39-40
6.2 All UML Diagrams	40-44
6.2.1 Data Flow Diagram	40-41
6.2.2 Use Case Diagram	41-42
6.2.3 Class Diagram	42
6.2.4 Sequence Diagram	43-44
6.3 Database Design	44-45
7. METHODOLOGY	46-48
7.1 Overall Pipeline	46
7.2 Steps of Methodology	46-47
7.3 Pseudocode	47-48
8. DATA SET DESCRIPTION	49-50
8.1 Dataset Source	49
8.2 Features	49
8.3 Data Size	49-50
8.4 Splitting of data	50
9. MODULE IMPLEMENTATION	51-56
9.1 Data Processing Module	51
9.2 Model Training & Evaluation Module	51
9.3 Prediction and Simulation Module	51
9.4 UI and API Module	52
9.5 Code Snippets	52-56
9.5.1 Server and API Setup	52-53
9.5.2 Core Engine and Microstructure	53-54
9.5.3 Artificial Intelligence Integration	54-55
9.5.4 Real-Time Communication and External Data	55-56
9.5.5 Frontend and User Interface	56

10. TESTING	57-61
10.1 Additional Test observations	58
10.2 Testing Pytest Snippets	58-60
10.2.1 Health and Frontend Tests	58-59
10.2.2 Stimulation API Tests	59
10.2.3 Agent API Tests	59-60
10.2.4 Execution Kernel Test	60
10.2.5 Research Workspace Tests	60-61
11. SCREENSHOTS	62-68
11.1 Landing Page	62
11.2 Research Workspace	62-63
11.3 Simulation Dashboard	64
11.4 Agent Analytics and Explainability	65-66
11.5 Live Market Intelligence	67-68
12. CONCLUSION & FUTURE WORK	69-70
12.1 Conclusion	69
12.2 Future Work	69-70
REFERENCES	71-72
APPENDIX I	73
APPENDIX II	79
APPENDIX III	81

I. ABSTRACT

Stock analysis platforms commonly emphasize historical charts, isolated prediction outputs, or manual decision support. Such systems often provide limited insight into market microstructure, trader behavior, execution constraints, and reproducible experimentation. In financial AI research, this creates a gap between algorithm development and the realistic study of how autonomous agents behave within a market environment.

The objective of StockAI v2.0 is to build a local-first, full-stack research platform for synthetic US equities market simulation. Rather than implementing only a single forecasting model, the project combines a hybrid market engine, AI-driven and deterministic agents, research dataset calibration, strategy evaluation workflows, and interactive web interfaces. The platform is designed to support scenario definition, run orchestration, order-book execution, explainability, event replay, and exportable research outputs.

The system is implemented using FastAPI for the backend, SQLite for persistent research storage, HTML/CSS/JavaScript with Chart.js for the frontend, and Pydantic for schema validation. The simulation kernel models a 25-stock US-equities universe distributed across six sectors. It includes session phases, correlated price walks, market events, financial reports, loans, circuit breakers, order latency, slippage, partial fills, and price-time-priority execution. Three agent categories are supported: LLM-backed behavioral agents, rule-based agents, and Python strategy-SDK agents. The platform also includes a live-market intelligence page that compares simulated market context with real-market snapshots fetched from Yahoo Finance.

Repository analysis shows that the project successfully implements a persistent research workflow centered on datasets, scenarios, experiments, bots, runs, evaluations, jobs, and run-scoped events. Actual API calls performed during analysis confirmed successful responses from `/health`, `/market/stocks`, `/simulation/status`, `/workspace/summary`, `/datasets/calibrate`, `/bots`, and `/evaluations`. Evaluation outputs demonstrated completed benchmarking records with aggregate metrics such as average PnL, Sharpe ratio, drawdown, and seed tracking. The overall result is a technically rich AI and data systems project that connects simulation, agent intelligence, analytics, and user interaction in a single research environment.

Keywords: Stock Market Simulation, Multi-Agent Systems, Artificial Intelligence, Market Microstructure, Algorithmic Trading, Large Language Models (LLMs), Reproducibility, Order-Book Execution.

II. LIST OF FIGURES

FIGURE NO.	NAME OF THE FIGURE	PAGE NO.
Fig. 4.1	System Architecture of the StockAI Framework	31
Fig. 4.2	Refined Architectural Flow of StockAI v2.0	32
Fig. 6.1	Architecture Diagram	39
Fig. 6.2	Data Flow Diagram	40
Fig. 6.3	Use Case Diagram	41
Fig. 6.4	Class Diagram	42
Fig. 6.5	Sequence Diagram	43
Fig. 6.6	Database Entity-Relationship Diagram	45
Fig. 11.1	Screenshot of Home Page	62
Fig. 11.2	Screenshots of Research Workspace	63
Fig. 11.3	Screenshot of Simulation Dashboard	64
Fig. 11.4	Screenshots of Agent Analytics & Explainability Panel	65-66
Fig. 11.5	Screenshots of Live Market Intelligence Page	67-68

III. LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
Table. 2.1	Summary of Literature on Agent-Based Modeling and LLM Financial Simulations	5
Table. 2.2	Relevance of StockAgent to StockAI v2.0	17
Table. 2.3	Comparison of Major Stock Forecasting Approaches	22
Table. 4.1	Key Functional Features Extracted from Repository	28
Table. 5.1	Functional Requirements	37
Table. 5.2	Non-Functional Requirements	38
Table. 6.1	Core Entities in Research Database	44
Table. 8.1	Seeded Market Universe Summary	49
Table. 8.2	Repository Data Artifacts in res/	50
Table. 10.1	Major Backend Modules	57-58

IV.ACRONYMS

ACRONYM	FULL FORM
AI	Artificial Intelligence +1
API	Application Programming Interface +1
ARIMA	AutoRegressive Integrated Moving Average
BE	Bachelor of Engineering
CD	Compact Disc
CI	Continuous Integration
CNN	Convolutional Neural Network
CSS	Cascading Style Sheets
CVaR	Conditional Value at Risk
DFD	Data Flow Diagram
DS	Data Science
ER	Entity-Relationship
ETF	Exchange Traded Fund
GRU	Gated Recurrent Unit
GUI	Graphical User Interface
HOD	Head of Department
HTML	HyperText Markup Language
IEEE	Institute of Electrical and Electronics Engineers
JSON	JavaScript Object Notation
LLM	Large Language Model +2
LSTM	Long Short-Term Memory +1
MACD	Moving Average Convergence Divergence
ML	Machine Learning
OHLCV	Open, High, Low, Close, Volume +1
PnL	Profit and Loss
RL	Reinforcement Learning
RSI	Relative Strength Index
SARIMA	Seasonal AutoRegressive Integrated Moving Average +1
SDK	Software Development Kit +1
UI	User Interface +1
UML	Unified Modeling Language
VaR	Value at Risk
VWAP	Volume Weighted Average Price
XGBoost	Extreme Gradient Boosting

1. INTRODUCTION

1.1 Introduction

The financial markets represent one of the most complex, dynamic, and data-rich environments in the modern world, making them a prime focus for artificial intelligence research. Traditionally, academic efforts in this domain have concentrated almost exclusively on predictive time-series forecasting, utilizing models such as ARIMA or LSTM to estimate future price points based solely on historical data. However, real-world trading is not merely a forecasting exercise; it is an interactive ecosystem governed by market microstructure, order book dynamics, execution latency, and the heterogeneous behavior of countless participating entities. StockAI v2.0 is introduced as a comprehensive solution to bridge the gap between isolated predictive modeling and realistic market execution. It is engineered as a hybrid synthetic US equities market simulation and research platform that allows analysts to test autonomous AI agents within a highly controlled, reproducible, and mechanically realistic trading environment.

1.2 Aim & Objectives

The primary aim of this project is to architect and develop a local-first, full-stack research platform capable of simulating a multi-agent synthetic stock market. To achieve this overarching goal, several key objectives have been established. First, the project seeks to implement a robust simulation kernel that accurately models market microstructure, including price-time priority order matching, latency, slippage, and correlated sector movements across a seeded universe of equities. Second, it aims to construct a diverse population of trading agents, encompassing Large Language Model-backed behavioral agents, deterministic strategy bots, and heuristic rule-based actors, thereby allowing for the study of emergent, heterogeneous market dynamics. Finally, the project is designed to ensure complete experimental reproducibility and behavioral explainability by persisting all datasets, scenarios, execution runs, and evaluation metrics within a structured SQLite database, all accessible through an interactive web-based orchestration interface.

1.3 Reason for Project

The fundamental motivation behind StockAI v2.0 stems from a distinct technological and academic gap in current financial research tools. While numerous platforms exist for historical chart analysis or commercial paper trading, they largely function as execution venues devoid of native AI research orchestration, or conversely, as rigid black-box predictors lacking execution realism. For researchers studying the behavioral economics of AI agents or assessing the robustness of trading algorithms under specific simulated market shocks, there is a significant lack of accessible, full-stack simulators that offer both multi-agent intelligence and microstructure fidelity. By developing this platform, the project provides a crucial laboratory environment where the complex transition from theoretical algorithm development to realistic, order-driven market execution can be seamlessly analyzed. Consequently, this project contributes a valuable, open-architecture experimentation tool to the broader fields of artificial intelligence and computational finance.

1.4 Problem Statement

Current financial AI research is often limited by a distinct disconnect between predictive model development and realistic market execution. Most academic projects focus narrowly on forecasting the next closing price using historical time-series data, ignoring crucial market microstructure elements such as latency, slippage, order matching, and the heterogeneous behavior of diverse market participants. Conversely, commercial algorithmic trading simulators offer realistic execution but lack the native integration of autonomous, AI-driven agents for continuous research and evaluation. Therefore, there is a critical need for a comprehensive, hybrid simulation platform that bridges this gap. The core problem this project addresses is the design and implementation of a local-first research environment that not only simulates a realistic, order-book-driven equities market but also orchestrates the continuous interaction of Large Language Model-backed, rule-based, and deterministic algorithmic agents while ensuring complete experimental reproducibility and behavioral explainability.

1.5 Scope

The scope of StockAI v2.0 encompasses the end-to-end development of a synthetic US equities market simulation framework integrated with a persistent research workspace. The platform includes the simulation of a seeded universe of 25 stocks distributed across six major sectors, incorporating realistic market dynamics such as regime transitions, event shocks, session phases, and price-time priority order execution. It supports the deployment, interaction, and evaluation of multi-agent populations, specifically accommodating LLM-based reasoning agents, deterministic strategy bots, and heuristic rule-based actors. Furthermore, the scope covers the implementation of a full-stack architecture utilizing a FastAPI backend, a SQLite database for the persistent management of datasets, scenarios, experiments, and run events, alongside interactive web interfaces for simulation orchestration and live-market intelligence comparison. The project explicitly excludes the creation of classical supervised training pipelines for direct financial forecasting, focusing instead on delivering the underlying simulation, orchestration, and evaluation architecture necessary to conduct advanced AI behavioral research.

1.6 Summary

This chapter provided an introductory overview of the StockAI v2.0 project, establishing the foundational context and highlighting the critical importance of artificial intelligence in modern financial systems. It examined the limitations of conventional, prediction-only forecasting models and articulated a clear problem statement addressing the academic and practical need for a hybrid, multi-agent simulation laboratory featuring realistic execution mechanics. Additionally, the scope of the project was thoroughly delineated to clarify the system boundaries, functional inclusions, and primary objectives of the implemented platform. With the background, problem, and scope clearly defined, the subsequent chapter will present a comprehensive literature survey, analyzing existing methodologies in time-series forecasting, deep learning, reinforcement learning, and agent-based computational finance to properly contextualize the unique architectural contributions of this proposed system.

2. LITERATURE SURVEY

2.1 Introduction

The literature related to StockAI v2.0 spans financial time-series forecasting, algorithmic trading, market microstructure, agent-based computational finance, behavioral finance, reinforcement learning, explainable AI, and full-stack research-platform engineering. A conventional stock-prediction project would normally focus only on forecasting models such as ARIMA, regression, random forests, LSTM, or transformer-based neural networks. StockAI v2.0 is broader than that. It is a hybrid synthetic US equities market simulation and research platform that uses FastAPI, SQLite, static web interfaces, configurable simulation runs, order-book execution, deterministic and AI-driven agents, and evaluation workflows.

Because of this wider scope, the literature survey must not treat the project as only a price-prediction system. The codebase shows that the project is closer to a research laboratory for simulated financial markets. It allows users to configure datasets, scenarios, experiments, bots, runs, evaluations, jobs, and run-scoped event streams. It also includes a synthetic 25-stock US equities universe, multi-agent behavior, execution latency, slippage, market orders, partial fills, queue-priority matching, circuit breakers, financial reports, loans, forum activity, explainability summaries, and live-market intelligence.

The main objective of this survey is therefore to review the research areas that motivate the project and to identify the limitations in existing approaches that StockAI v2.0 attempts to address. Classical forecasting models are reviewed first because they form the historical foundation of financial prediction. Machine learning and deep learning methods are then discussed as more flexible predictive techniques. Reinforcement learning is reviewed because it models trading as a sequential decision process. Agent-based computational finance is discussed in greater depth because it is the closest match to the implemented platform. Finally, market microstructure, behavioral finance, explainability, live-market intelligence, and system architecture literature are reviewed to connect academic concepts to the actual implementation.

The key theme emerging from the literature is that prediction alone is not enough for serious financial AI research. A model may produce a price estimate or a buy/sell signal, but that does not answer how the trade is executed, how agents interact, how market regimes affect behavior, how

results can be reproduced, or why a decision was made. StockAI v2.0 is motivated by this gap. It provides a controlled market environment where strategies and agents can be tested under configurable conditions, with persistent evidence stored for later analysis.

S.No	Year	Title	Strategy/Method	Dataset	Advantages
1	2024	StockAgent (Zhang et al., 2024)	LLM-based multi-agent simulation	stock market data	Realistic trading behavior
2	2023	Generative Agents: Interactive Simulacra of Human Behavior (Park et al., 2023)	LLM-driven agent simulation with memory and planning	Custom social simulation environment	Proves LLM agents exhibit emergent, consistent behaviour across time
3	2024	FinAgent: A Multimodal Foundation Agent for Financial Trading (Liu et al., 2024)	Multimodal LLM with market data + news fusion	US stock market data + financial news corpus	Demonstrates LLM reasoning on real financial signals with explainability
4	2023	ReAct: Synergizing Reasoning and Acting in Language Models (Yao et al., 2023)	Chain-of-thought reasoning + action loop (ReAct)	Multiple reasoning and interactive benchmarks	Framework for LLM agents that reason before acting - directly informs StockAI agent prompt design
5	2001	A Builder's Guide to Agent-Based Financial Markets (LeBaron, 2001)	Agent-based modelling of price dynamics	Synthetic financial market with heterogeneous agents	Foundational proof that synthetic markets reproduce fat tails and volatility clustering
6	2009	The Economy Needs Agent-Based Modelling (Farmer & Foley, 2009)	Macro agent-based economic simulation	Theoretical / calibrated economic data	Argues ABM outperforms equilibrium models for systemic risk - justifies StockAI's simulation approach
7	2003	Market Microstructure: Orders, Prices and Quotes (Harris, 2003)	Theoretical analysis of limit order books, spreads, and price formation	NYSE and NASDAQ historical trade data	Theoretical foundation for StockAI's order book, session phases, slippage, and spread mechanics
8	2018	Measuring the Intrinsic Dimension of Objective Landscapes — Seed Reproducibility in ML Research (Li et al., 2018 / extended)	Reproducibility analysis and seeding strategies in machine learning experiments	Standard ML benchmarks	Informs StockAI's reproducibility design - <code>random.seed()</code> + <code>numpy.random.seed()</code> at run start guarantees identical outputs per seed

Table 2.1 Summary of Literature on Agent-Based Modeling and LLM Financial Simulations

2.2 Classical Time-Series Forecasting

Classical time-series forecasting provides the earliest formal foundation for stock-market prediction. Models such as autoregressive moving average (ARMA), autoregressive integrated moving average (ARIMA), seasonal ARIMA, exponential smoothing, and moving-average rules attempt to explain future values from past observations and error structures. These models are attractive because they are mathematically clear, computationally efficient, and useful as baselines. In academic projects, ARIMA is often used as a benchmark against which more complex machine learning or deep learning models are compared.

ARIMA models are built on the assumption that a time series can be made stationary through differencing and then modeled using autoregressive and moving-average components. This is useful when a series has stable linear dependence. However, financial markets are rarely stable for long periods. Asset returns are noisy, nonstationary, and affected by macroeconomic events, investor behavior, market liquidity, and sudden information shocks. As a result, ARIMA-style models can be useful for short-horizon baseline comparisons but are limited when the research goal is to understand interactive market dynamics.

Volatility models such as ARCH and GARCH were developed to address time-varying variance. They are important because financial returns often show volatility clustering: large moves tend to be followed by large moves, and quiet periods tend to be followed by quiet periods. GARCH-type models are more realistic than simple constant-variance models, but they still focus mainly on statistical properties of returns rather than on the behavior of market participants. They do not naturally explain how heterogeneous agents, order-book queues, sentiment shifts, or execution frictions generate observed outcomes.

For StockAI v2.0, classical time-series models are best understood as baseline literature rather than as the core implementation. The repository does not implement ARIMA or GARCH training pipelines. Instead, it uses a synthetic market engine with events, regimes, correlated price walks, agent behavior, and execution mechanics. This gives the platform a different purpose. It can eventually host ARIMA or GARCH baselines as strategy modules, but its current contribution is a simulation and experimentation environment.

The limitation of classical models supports the need for StockAI's broader design. A time-series model may forecast a price path, but it does not show how autonomous traders react to news, how order matching affects fills, or how a scenario changes portfolio results across multiple seeds.

StockAI v2.0 addresses these missing dimensions by focusing on simulated market process, not only numerical prediction.

2.3 Stylized Facts and Financial Data Complexity

An important body of financial literature studies the statistical properties of asset returns. Cont's work on stylized facts is especially relevant because it summarizes recurring empirical properties across markets, including heavy-tailed return distributions, volatility clustering, absence of simple linear autocorrelation in returns, and nonlinear dependence in volatility. These properties explain why stock forecasting is difficult and why naive statistical assumptions are often unreliable.

Stylized facts matter for StockAI v2.0 because a simulator should not merely generate smooth random prices. If simulated prices behave too cleanly, trading strategies tested inside the environment may appear stronger than they would be in real markets. A research platform should therefore include volatility regimes, shocks, sector correlations, and abnormal events. The StockAI simulation engine includes correlated price walks, market events, sector drifts, regime transitions, and circuit breakers, which are practical attempts to move beyond oversimplified random-walk demonstrations.

Financial data is also affected by nonstationarity. The relationship between indicators and returns may change during bull markets, bear markets, crises, earnings seasons, inflation shocks, or sector rotations. A supervised model trained on one period can fail when the regime changes. This is a central problem for financial AI, and it motivates scenario-based testing. StockAI's use of datasets, scenarios, experiments, and run records reflects this literature concern: results should be tied to the conditions under which they were produced.

Another complexity is cross-asset dependence. Stocks in the same sector often move together because they share economic drivers, investor flows, and news exposure. A single-stock predictor that ignores sector movement can misinterpret broad market drift as stock-specific strength. StockAI v2.0 addresses this by organizing a 25-stock universe across sectors and computing market analytics such as sector indices, breadth, and benchmark return.

The literature on stylized facts also reveals a difference between statistical realism and behavioral realism. A model can reproduce heavy tails but still fail to model trader decision-making. Conversely, a behavioral simulator can include agents but produce unrealistic returns if it is not calibrated. StockAI v2.0 sits between these goals. It uses calibrated priors and configurable

research records while allowing agent behavior and market events to generate evolving outcomes. This makes it suitable for educational and experimental finance research, even if it is not a replacement for exchange-grade market replay.

2.4 Machine Learning for Stock Prediction

Machine learning introduced a major shift in stock-market prediction by allowing models to learn nonlinear relationships from engineered features. Common approaches include linear regression, ridge regression, lasso regression, support vector regression, decision trees, random forests, gradient boosting, support vector machines, k-nearest neighbors, and neural networks. These models are often trained on historical price, volume, technical indicators, macroeconomic data, and sentiment features.

Regression-based models attempt to predict a continuous value such as the next closing price, future return, or volatility. Classification-based models simplify the task into categories such as up, down, neutral, buy, sell, or hold. Classification is often more practical for trading decisions because exact price prediction is extremely difficult. Directional accuracy, risk-adjusted return, drawdown, and transaction costs are usually more meaningful than raw error metrics alone.

Tree-based models and ensemble methods are popular because they can capture nonlinear feature interactions and are easier to train than deep learning models. Random forests reduce overfitting through averaging across multiple trees, while gradient boosting methods build strong predictors by combining weak learners. In finance, these methods can perform well when features are carefully engineered. However, they are still vulnerable to overfitting, look-ahead bias, data leakage, regime shifts, and unrealistic backtesting.

Machine learning literature is relevant to StockAI v2.0 in two ways. First, it provides future strategy candidates. A trained classifier could be wrapped as a strategy agent that observes the market state and emits order intents. Second, it highlights the importance of evaluation design. A model's historical accuracy is not enough if the trading environment ignores spread, slippage, latency, and partial fills. StockAI's evaluation framework, order-book logic, and research persistence address precisely these practical issues.

The current repository does not contain a complete supervised ML training pipeline. This is not a weakness if the project is correctly described. StockAI v2.0 is not mainly a machine learning predictor. It is a platform into which ML strategies can be inserted, tested, compared, and

explained. This distinction is important for academic honesty. The project contributes infrastructure for reproducible agent evaluation rather than claiming a single best predictive model.

2.5 Deep Learning and Sequence Models

Deep learning became prominent in stock prediction because financial data is sequential, noisy, and nonlinear. Recurrent neural networks (RNNs), long short-term memory networks (LSTMs), gated recurrent units (GRUs), convolutional neural networks (CNNs), hybrid CNN-LSTM models, attention mechanisms, and transformers have all been applied to financial time series. LSTM is especially common in academic stock-prediction projects because it is designed to learn temporal dependencies while reducing the vanishing-gradient problem found in basic RNNs.

Fischer and Krauss demonstrated the use of LSTM networks for financial-market prediction and compared them with other methods. Their work is frequently cited because it treats LSTM as a serious candidate for directional market prediction rather than only as a demonstration model. Other studies use LSTM models with technical indicators, sentiment scores, news embeddings, or market-index features. Hybrid neural architectures attempt to combine local pattern extraction with long-term temporal memory.

Despite their strengths, deep learning models face major limitations in finance. They require large and carefully cleaned datasets, are sensitive to preprocessing, and can overfit historical patterns that disappear in future regimes. They are also often difficult to interpret. A model may output a prediction, but the user may not understand which market context influenced the result. In financial decision support, opacity is a serious concern because users need trust, auditability, and risk awareness.

Deep learning literature also tends to focus on prediction more than execution. Many papers report accuracy, mean squared error, or directional performance but do not model realistic order placement, queue priority, transaction costs, market impact, or slippage. A strategy can look profitable when evaluated on closing prices but fail when orders must be executed through a market mechanism. This gap is one of the reasons platforms like StockAI v2.0 are valuable.

StockAI v2.0 does not implement LSTM or transformer training in its current codebase. However, it is compatible with future deep learning integration. A deep model could become a strategy-SDK agent, a signal provider for behavioral agents, or a forecasting panel in the research workspace. The platform's strongest immediate contribution is that it can evaluate such models inside a richer

simulated market context rather than treating prediction as an isolated spreadsheet task.

2.6 Sentiment, News, and Event-Driven Market Models

Financial markets respond not only to past prices but also to information. Earnings reports, macroeconomic announcements, analyst upgrades, geopolitical events, product launches, scandals, regulatory actions, and social-media trends can all influence stock prices. This has led to a large literature on sentiment analysis and event-driven forecasting. Natural language processing models are used to transform news articles, filings, social posts, and forum discussions into features that can support prediction or trading decisions.

Early sentiment systems used dictionaries and rule-based polarity scoring. Later systems used machine learning classifiers and word embeddings. More recent systems use transformer-based language models that can extract context, entity relationships, and event types from text. In finance, however, sentiment is difficult because language can be domain-specific. A phrase that is positive in ordinary language may be neutral or negative in a financial context. Market reaction also depends on expectations: good earnings may cause a stock to fall if investors expected even better results.

StockAI v2.0 includes event-oriented simulation concepts rather than a full-scale news-ingestion NLP pipeline. The simulation engine generates market events, financial reports, and forum posts. Behavioral agents receive market context and news-like signals, then make decisions. This design reflects the importance of information flow in markets while remaining manageable for a local-first academic platform.

Event-driven models also connect to scenario testing. A platform should allow researchers to ask questions such as: What happens if the technology sector receives a negative shock? How do high-risk agents behave during volatility? Does a mean-reversion strategy survive during a regime transition? StockAI's scenario and experiment records are aligned with this kind of research.

The literature suggests that event-driven intelligence should be evaluated differently from ordinary prediction. It is not enough to test whether a model predicts the next price. Researchers should examine the chain from event to perception, decision, order, execution, portfolio impact, and explanation. StockAI v2.0 supports this chain through agent decisions, order-book execution, run events, analytics, and exportable research bundles.

2.7 Reinforcement Learning in Trading

Reinforcement learning (RL) models decision-making as an agent interacting with an environment. The agent observes a state, chooses an action, receives a reward, and updates its policy to maximize long-term return. This is naturally attractive for trading because trading is sequential. A trader must decide not only what to buy or sell, but when to act, how much to trade, how to manage risk, and when to exit.

RL literature in financial markets includes value-based methods such as Q-learning and deep Q-networks, policy-gradient methods, actor-critic methods, and more recent algorithms such as proximal policy optimization. Researchers apply RL to portfolio allocation, execution optimization, market making, and directional trading. Meng and Khushi's survey notes that RL can combine prediction and portfolio construction into an integrated decision problem while accounting for constraints such as transaction costs and liquidity.

The main challenge is environment realism. RL agents can exploit weaknesses in a simulator. If a simulated environment ignores transaction costs, market impact, latency, or liquidity constraints, the trained policy may learn unrealistic behavior. Reward engineering is also difficult. A reward based only on profit may encourage excessive risk, while a reward with too many penalties may prevent learning. Financial RL therefore depends heavily on careful simulator design and robust evaluation.

StockAI v2.0 does not currently implement RL training, but it contains several RL-ready ingredients. It has structured observations, agent action loops, market state, price history, events, order intents, fills, portfolio updates, evaluation metrics, and reproducible runs. A future RL extension could use the simulation loop as an environment and implement policies as strategy agents. The existing research store could persist training configurations, evaluation seeds, and performance records.

The RL literature supports StockAI's emphasis on reproducibility. A single profitable run is not meaningful. Strategies must be evaluated across multiple seeds, scenarios, and market regimes. The platform's evaluations, jobs, run records, and aggregate metrics such as PnL, Sharpe ratio, and drawdown are consistent with this requirement. In this sense, StockAI provides infrastructure that would be valuable even before RL algorithms are added.

2.8 Algorithmic Trading and Strategy Evaluation

Algorithmic trading literature focuses on rules and models that generate automated trading decisions. Strategies may be based on momentum, mean reversion, pairs trading, statistical arbitrage, market making, VWAP execution, trend following, or machine learning signals. A serious strategy evaluation pipeline must go beyond signal accuracy and include transaction costs, position limits, risk controls, drawdown, execution quality, and benchmark comparison.

Momentum strategies assume that recent price trends may continue for some period. Mean-reversion strategies assume that deviations from a fair value or moving average may correct. VWAP-related strategies attempt to execute trades near the volume-weighted average price. These classical strategies are useful baselines because they are understandable and can be compared against more complex AI agents.

StockAI v2.0 includes a strategy SDK with mean-reversion and VWAP benchmark logic. This is important because it allows deterministic strategies to coexist with behavioral and LLM-backed agents. A research platform should not only include intelligent agents; it should also include simple baselines. Without baselines, it is difficult to know whether a complex system is genuinely useful or only visually impressive.

Evaluation metrics are central to algorithmic trading. Profit and loss alone can be misleading because a high-return strategy may also take unacceptable risk. Sharpe ratio, maximum drawdown, win rate, turnover, exposure, and execution cost provide a fuller picture. StockAI’s evaluation reports include aggregate metrics such as average PnL, average Sharpe ratio, and average maximum drawdown. These align with common trading evaluation practices.

The literature also warns against overfitting and backtest bias. Strategies may look profitable because of data snooping, parameter tuning, or accidental future information. StockAI’s reproducible seeds and scenario records are useful safeguards because they make experiments easier to repeat and compare. The platform can be extended further with walk-forward testing, train-test regime splits, and transaction-cost stress tests.

2.9 Agent-Based Computational Finance

Agent-based computational finance (ABCF) studies financial markets as systems of interacting agents. Instead of assuming one representative rational investor, agent-based models include heterogeneous participants with different strategies, risk preferences, information, time horizons,

and behavioral biases. LeBaron’s survey is a foundational reference in this area because it explains how computational models can reveal the importance of investor heterogeneity in financial settings.

ABCF is highly relevant to StockAI v2.0. The project models a market as an environment where multiple agents observe state, receive news, form intentions, and submit orders. The agents are not identical. The repository supports LLM-backed behavioral agents, rule-based agents, and strategy-SDK agents. This is much closer to agent-based finance than to a single-model forecasting system. Agent-based models are useful because many market phenomena are emergent. Price movement, volatility clustering, liquidity stress, herding, bubbles, and crashes can arise from interactions among participants rather than from one isolated signal. A simulator can therefore be used to study “what-if” scenarios that are difficult to test in real markets. For example, researchers can examine how an event shock affects agents with different risk appetites or how liquidity changes when many agents submit similar orders.

The limitations of agent-based models are also important. Their realism depends on calibration, parameter choices, and behavioral assumptions. If agent rules are too simple, the market may not behave realistically. If they are too complex, the simulator may become difficult to interpret. Validation against real market data is challenging. StockAI addresses some of these concerns through calibrated datasets, scenario records, explainability summaries, and exportable run bundles, but future work should strengthen empirical calibration.

The ABCF literature provides the strongest academic justification for StockAI v2.0. The project is best described as an AI-enabled agent-based market simulation and research platform. It provides a controlled environment for studying agents, execution, events, and reproducible experiments.

2.10 Market Microstructure and Limit Order Ideas

Market microstructure studies how trading mechanisms affect prices, liquidity, and execution. It examines bid-ask spreads, order types, limit order books, market orders, queue priority, latency, partial fills, market impact, and price discovery. This literature is essential for StockAI v2.0 because the project includes an order-book execution engine rather than only abstract buy/sell decisions.

Limit order books are central to modern electronic markets. A limit order specifies a price and quantity, while a market order seeks immediate execution against available liquidity. Orders are usually matched according to price-time priority: better prices execute first, and earlier orders at the same price have priority. Gould and coauthors survey limit order books and emphasize that order-book systems are complex and that many models fail to reproduce real empirical properties fully.

Many academic stock-prediction systems ignore microstructure. They assume a trader can buy or sell at the observed closing price without spread, delay, or liquidity constraints. This assumption can make strategies appear more profitable than they are. Real trading faces friction. Large orders may be partially filled. Market orders may receive worse prices than expected. Delays may cause missed opportunities. Thin liquidity may increase price impact.

StockAI v2.0 directly incorporates several microstructure concepts. The *OrderBook* and *SimulationLoop* modules support market and limit orders, price-time-priority matching, self-trade avoidance, partial fills, latency steps, slippage adjustment, spread bands, market depth, and circuit-breaker style halts. These features make the platform more realistic than a chart-only simulator.

The literature suggests that execution realism is not optional for trading research. A strategy is only meaningful if its signals can be translated into orders under realistic constraints. StockAI's execution kernel therefore provides an important bridge between AI decision-making and market process. Future enhancements could include more detailed queue modeling, exchange fee schedules, hidden liquidity, order cancellations, and historical order-book replay.

2.11 Behavioral Finance and Trader Psychology

Behavioral finance challenges the idea that investors always act rationally. Kahneman and Tversky's prospect theory shows that people evaluate gains and losses asymmetrically and often display risk aversion for gains and risk seeking for losses. Other behavioral concepts include overconfidence, anchoring, herding, loss aversion, confirmation bias, recency bias, and disposition effect. These biases are important because real markets are influenced by human psychology.

StockAI v2.0 includes behavioral agents whose actions can be influenced by persona, risk appetite, conviction, active biases, and market context. This design is significant because it acknowledges that market simulation should include more than mechanical technical indicators. A fearful trader, an overconfident trader, and a momentum-chasing trader may respond differently to the same

event.

Behavioral finance is especially useful in simulation because biases can be controlled and studied. Researchers can ask whether herding increases volatility, whether loss aversion delays selling, or whether overconfidence increases turnover. In real markets, these effects are hard to isolate. In a simulator, they can be represented through agent attributes and scenario settings.

The project's explainability layer also relates to behavioral finance. It logs decision details, active biases, conviction, reasoning summaries, and agent analytics. This allows users to inspect not only what an agent did but why it acted. In educational settings, this is valuable because students can connect abstract behavioral-finance concepts to visible simulated outcomes.

The limitation is that simulated psychology is an approximation. LLM-backed or rule-based personas do not perfectly represent real investors. However, the goal of an academic simulator is not to reproduce every human detail. The goal is to create a controlled environment where behaviorally meaningful assumptions can be tested. StockAI v2.0 provides such an environment and can be extended with richer behavioral models in future work.

2.12 Large Language Models and Generative Agents

Large language models (LLMs) have introduced new possibilities for agent simulation. Instead of relying only on fixed rules, LLM-backed agents can generate natural-language reasoning, interpret context, and produce varied decisions. Research on generative agents shows that language models can simulate believable human-like behavior in interactive environments when given memory, reflection, and planning structures.

In financial systems, LLMs can support news interpretation, decision explanations, scenario summaries, research assistance, and conversational interfaces. However, they also introduce risks. LLM outputs can be inconsistent, overly confident, or unsupported by data. For trading applications, this means LLMs should not be treated as unquestionable decision engines. They require guardrails, structured prompts, validation, and fallback behavior.

StockAI v2.0 uses optional provider-backed intelligence through Groq, OpenAI, and Google Gemini configuration, while also supporting deterministic or mock behavior. This is a practical architecture because the platform remains usable without external API keys. LLM-backed agents enrich the simulation by producing behavior and explanations, but deterministic agents and strategy agents provide stability and baselines.

The literature on generative agents supports StockAI’s design choice to combine personality, context, and decision logs. A financial agent should receive structured market state, recent events, portfolio context, and constraints before acting. The repository’s prompt factory and agent logic reflect this idea by building market-aware prompts and recording decision metadata.

For academic reporting, it is important to state that StockAI does not train its own LLM. It integrates optional LLM providers as part of agent reasoning and chat support. This distinction avoids overstating the project. The contribution is not a new language model but the integration of language-model-driven behavior into a reproducible market simulation and research platform.

2.13 StockAgent and LLM-Based Multi-Agent Trading Simulation

StockAgent, proposed by Zhang et al. in 2024, is one of the most directly relevant recent works for StockAI v2.0 because it studies stock trading through an LLM-based multi-agent simulation framework. The paper, titled “When AI Meets Finance (StockAgent): Large Language Model-based Stock Trading in Simulated Real-world Environments,” introduces a system where multiple AI agents simulate investor trading behavior in response to stock-market data and external factors. These external factors include macroeconomic changes, policy shifts, company fundamentals, emergencies, benchmark interest rates, and bulletin-board discussions.

The central idea of StockAgent is that LLM-driven agents can be used not only for prediction but also for behavioral simulation. Instead of asking a model only to forecast a price, the system places LLM agents inside a simulated trading environment and observes how they make decisions. This is important because real markets are affected by the behavior of many interacting participants. StockAgent therefore aligns with the broader agent-based computational finance literature while adding recent advances in language-model reasoning.

StockAgent’s design includes multiple stages of trading. The pre-trading stage prepares agents using market conditions, interest rates, and financial events. The trading stage allows transactions and account updates. The post-trading stage supports reflection, strategy sharing, and future planning. The inclusion of a bulletin board system is especially interesting because it models how investor opinions can influence other investors. This makes StockAgent more realistic than systems where each agent acts in isolation.

Another important contribution of StockAgent is its attention to test-set leakage. In financial applications, LLMs may already contain knowledge about historical events or market data from

pretraining. If the model relies on memorized information, simulation results become unreliable. Zhang et al. discuss methods to reduce the influence of prior knowledge so that agents respond to the supplied simulation context. This issue is highly relevant for any LLM-based stock simulation, including StockAI v2.0, because provider-backed agents may carry broad world knowledge that must be controlled through prompt design, scenario isolation, and evaluation discipline.

StockAgent evaluates different LLMs, including GPT and Gemini models, within its trading framework. The purpose is not merely to find a profitable model but to investigate whether LLM agents can produce realistic trading behavior under different external conditions. This research direction is valuable because it shifts attention from single-number prediction accuracy toward behavioral reliability, profitability effects, and market response patterns.

2.14 Relevance of StockAgent to StockAI v2.0

StockAgent is highly relevant to StockAI v2.0 because both systems treat stock-market intelligence as a multi-agent simulation problem. The user-provided comparison can be summarized as follows:

Year	System	Technique	Data/Environment	Key Outcome
2024	StockAgent (Zhang et al., 2024)	LLM-based multi-agent simulation	Stock market data	Realistic trading behavior

Table 2.2 Relevance of StockAgent to StockAI v2.0

This comparison helps position StockAI v2.0 within recent literature. StockAgent demonstrates that LLM-based investors can be studied inside simulated market environments. StockAI v2.0 follows a related motivation but implements its own local-first platform with FastAPI APIs, static web dashboards, SQLite research persistence, strategy agents, behavioral agents, order-book execution, run-scoped events, and live-market intelligence. In other words, StockAgent is a research reference for the LLM-agent simulation idea, while StockAI v2.0 is a full-stack implementation that combines simulation, evaluation, explainability, and user-facing research workflows.

There are several design parallels. First, both systems emphasize external factors. StockAgent

studies how macroeconomics, fundamentals, policies, emergencies, and BBS discussions affect agent decisions. StockAI v2.0 includes market events, financial reports, forum activity, scenario records, and live-market context. Second, both systems model investor-like agents rather than only technical indicators. StockAgent uses LLM-driven AI agents, while StockAI supports LLM-backed behavioral agents, rule-based agents, and strategy-SDK agents. Third, both systems value behavioral realism. The goal is not only to output a recommendation but to simulate plausible trading responses.

StockAI v2.0 also extends the discussion in directions that are important for project implementation. Its order-book engine models market and limit orders, price-time priority, partial fills, latency, slippage, and market depth. This supports execution-aware simulation, which is essential when evaluating whether agent decisions can become actual trades. Its research store records datasets, scenarios, experiments, bots, runs, evaluations, jobs, and run events. This supports reproducibility and makes the platform useful for repeated academic experimentation.

The StockAgent paper also identifies risks that StockAI should continue addressing. LLM agents may show model-specific tendencies, inconsistent behavior, or hidden reliance on prior knowledge. Therefore, StockAI should treat LLM-backed behavior as one agent category rather than the only source of truth. Deterministic rule-based agents and strategy agents provide necessary baselines. Explainability logs, decision traces, and scenario metadata help users understand whether agent behavior is meaningful or merely a prompt artifact.

Overall, StockAgent strengthens the academic motivation for StockAI v2.0. It shows that LLM-based multi-agent trading simulation is an active and relevant research direction in 2024. StockAI v2.0 contributes to this direction by building a practical platform where similar ideas can be explored through configurable simulations, persistent experiments, order-book execution, explainability dashboards, and live-market comparison.

2.15 Research Platforms and Experiment Reproducibility

A major weakness in many stock-prediction projects is poor reproducibility. Results may depend on hidden preprocessing, undocumented random seeds, changing datasets, manually tuned parameters, or unrecorded assumptions. In finance, reproducibility is especially important because small changes in evaluation design can produce large differences in apparent performance.

Research platforms address this problem by storing datasets, experiment configurations, model

versions, run records, metrics, logs, and artifacts. A good platform allows researchers to compare runs, export evidence, rerun scenarios, and trace conclusions back to specific inputs. This is common in machine learning operations and increasingly necessary in financial AI research.

StockAI v2.0 implements a research store backed by SQLite. It manages datasets, scenarios, experiments, agent populations, bots, runs, run events, evaluations, and jobs. This is one of the strongest system-level contributions in the repository. Instead of treating each simulation as a temporary browser session, the platform stores structured records that can be inspected later.

The research workspace also reflects reproducibility principles. It allows users to inspect dataset readiness, scenario readiness, evaluation status, and latest completed evaluations. API routes expose resources such as */datasets*, */scenarios*, */experiments*, */bots*, */runs*, */evaluations*, and run-scoped event streams. This creates a research workflow rather than only a demonstration UI.

The literature on reproducible finance emphasizes that experiments should include risk metrics, costs, seeds, and configuration snapshots. StockAI already supports many of these ideas and can be extended with stronger metadata such as git commit hash, dependency version, random-state details, and exported environment settings. The current design is a strong foundation for academic evaluation because it treats simulation as evidence-producing research work.

2.16 Explainable AI in Financial Decision Support

Explainable AI (XAI) is important in finance because decisions can affect money, risk, and trust. Users need to understand why a model or agent recommended an action. Black-box predictions are difficult to accept when they cannot be audited. In regulated or high-stakes environments, explainability also supports accountability and debugging.

Financial explainability has several levels. At the model level, techniques such as feature importance, SHAP values, attention visualization, or counterfactual explanations can show what influenced a prediction. At the strategy level, explanations can describe why a trade was entered or exited. At the simulation level, explanations can show how market events, agent biases, and order-book conditions produced outcomes.

StockAI v2.0 focuses on agent and simulation explainability. The behavioral-agent layer records decision logs, active biases, conviction, action type, memo fields, and analytics. The frontend includes explainability dashboards and agent analytics surfaces. This is appropriate for the project's architecture because the main decisions are made by agents inside a simulated market.

Explainability also helps detect simulation problems. If agents all behave identically, explanations may reveal poor prompt diversity or overly rigid rules. If a strategy produces high returns through unrealistic fills, execution logs can expose the issue. Therefore, explainability is not only a user-interface feature; it is a research and debugging tool.

Future XAI improvements could include structured reason codes, counterfactual replay, “what changed since last action” summaries, feature-attribution for strategy agents, and side-by-side explanation comparison across agent types. Even in its current form, StockAI’s explainability layer distinguishes it from ordinary stock dashboards, which often display signals without decision provenance.

2.17 Live-Market Intelligence and Synthetic-to-Real Comparison

Synthetic simulation is useful because it provides control, repeatability, and safe experimentation. However, it can become detached from real market conditions if not compared with live or historical data. Literature on financial simulation emphasizes calibration and validation: a model should be checked against real-world patterns, even if it is not intended to predict the exact future. StockAI v2.0 includes a live-market intelligence surface that fetches real-market snapshots using a Yahoo Finance chart endpoint through *httpx*. The live-market page compares market context, sector pulse, movers, watchlist items, and simulator context. This feature helps bridge the synthetic environment with external market reality.

Live-market comparison is valuable for education and research. A student can observe how the synthetic technology sector behaves under a scenario and then compare it with real market movement. A researcher can use live snapshots as inspiration for scenario creation or calibration. A bot builder can compare simulated strategy behavior against current market conditions without connecting to a broker.

The limitation is that live snapshots are not the same as full historical market data or exchange-grade feeds. Yahoo Finance data is useful for broad context, but it is not designed as a high-frequency trading data source. StockAI handles this appropriately by presenting live intelligence as a comparative layer rather than as a guaranteed execution feed.

Future work could include historical OHLCV ingestion, sector ETF calibration, benchmark replay, macroeconomic event imports, and configurable data providers. The current live-market layer is still important because it prevents the simulator from feeling isolated. It makes the platform more

useful as a research and teaching tool by placing synthetic experiments beside real market signals.

2.18 Full-Stack Architecture for Financial AI Tools

Financial AI platforms require more than models. They need backend APIs, schema validation, persistence, frontend visualization, testing, deployment, and operational configuration. Poor software architecture can make an otherwise interesting model difficult to use, reproduce, or extend. Therefore, system-design literature and best practices are relevant to StockAI v2.0.

The project uses FastAPI for backend routes, Pydantic for domain schemas, SQLite for research persistence, static HTML/CSS/JavaScript for frontend pages, Chart.js for visualization, Pytest for testing, and optional provider SDKs for AI integration. This stack is appropriate for a local-first academic platform because it is lightweight, understandable, and deployable without a complex frontend build system.

The backend separates API modules for market, simulation, agents, data, chat, research, live-market, and websocket functionality. The engine layer contains simulation and order-book behavior. The core layer contains analytics, research storage, prompt construction, job management, LLM provider wrappers, and live-market services. The model layer defines typed records and domain entities. This modular structure makes the system easier to reason about.

Frontend architecture is also important. StockAI provides multiple user-facing surfaces: a landing page, a research workspace, a simulator dashboard, and a live-market intelligence page. This separation reflects different user workflows. A researcher needs experiment management, a simulator user needs live state and controls, and a market observer needs real-time context.

The literature on software engineering for AI systems emphasizes observability, testing, and modularity. StockAI includes tests for critical API paths and execution-kernel behavior. This supports confidence in the platform’s core workflow. Future architectural improvements could include authentication, role-based access, migration management, stronger frontend componentization, and cloud-hosted multi-user experiment tracking.

2.19 Comparison of Existing Approaches

The existing literature and practical systems can be grouped into several categories. Charting tools provide visual indicators and price history but usually do not support agent simulation or reproducible experiments. Forecasting systems predict prices or directions but often ignore order

execution and multi-agent behavior. Backtesting engines evaluate strategies on historical data but may not include heterogeneous behavioral agents. Agent-based simulators model interactions but may lack modern web interfaces and persistent research workflows.

StockAI v2.0 combines elements from these categories. It includes market visualization, strategy evaluation, agent-based simulation, research storage, explainability, and live-market comparison. Its main value lies in integration. Rather than maximizing sophistication in one narrow model, it creates a complete experimental environment for financial AI behavior.

The following comparison summarizes the relationship:

Approach	Main Contribution	Common Limitation	StockAI v2.0 Relationship
Classical forecasting	Interpretable baseline models	Weak under nonlinear regimes	Useful future benchmark
Machine learning prediction	Nonlinear feature learning	Often ignores execution	Can be integrated as strategy agents
Deep learning	Temporal representation	Data hungry and opaque	Future forecasting extension
Reinforcement learning	Sequential decision optimization	Needs realistic environment	StockAI can become an RL environment
Backtesting engines	Strategy replay and metrics	Often historical-data focused	StockAI adds synthetic scenarios and agents
Agent-based simulation	Heterogeneous market behavior	Calibration difficulty	Closest match to StockAI
Chart dashboards	Usable visualization	Limited research depth	StockAI includes dashboards plus persistence

Table 2.3 Comparison of Major Stock Forecasting Approaches

This comparison shows that StockAI's contribution is not a replacement for all prior methods. It is a platform that can host and compare them. A future ARIMA model, LSTM model, RL policy, or rule-based strategy could be inserted into the same research workflow. This extensibility is academically valuable because it turns the project into a foundation for continued experimentation.

2.20 Identified Literature Gaps

The literature reveals several gaps that motivate StockAI v2.0. First, many forecasting studies focus on predictive accuracy but do not show how predictions translate into executable trades. They may report mean squared error or directional accuracy without accounting for spread, slippage, liquidity, latency, or partial fills. This weakens practical relevance.

Second, many systems lack agent heterogeneity. Real markets contain institutions, retail traders, market makers, speculators, cautious investors, overconfident investors, and algorithmic strategies. A single representative model cannot capture the diversity of market behavior. Agent-based literature addresses this gap, and StockAI implements multiple agent categories.

Third, many academic prototypes lack reproducibility. They may not store dataset versions, scenario configurations, random seeds, experiment records, or evaluation artifacts. StockAI addresses this through SQLite-backed research records and run-scoped events.

Fourth, explainability is often underdeveloped. A user may see a signal without understanding the reasoning, bias, or market state behind it. StockAI records agent decisions and supports explainability dashboards, making it easier to inspect behavior.

Fifth, synthetic simulators can become isolated from real markets. StockAI partially addresses this by adding live-market intelligence and external snapshot comparison. This does not make the simulator fully calibrated, but it provides an important connection between synthetic experiments and current market context.

The main remaining gaps for StockAI are also clear. The platform would benefit from historical-data ingestion, stronger empirical calibration, built-in ARIMA/LSTM/RL baselines, walk-forward evaluation, richer transaction-cost modeling, and more formal validation against real market stylized facts. These gaps provide a strong future-work roadmap.

2.21 Relevance to StockAI v2.0 Modules

The literature can be mapped directly to the repository modules. Classical forecasting and machine learning literature relate to future strategy extensions in *backend/app/sdk/strategy.py* and *backend/app/agents/strategy_agent.py*. Although these models are not currently trained in the repository, the strategy interface provides a natural place to integrate them.

Agent-based computational finance maps to *backend/app/agents/behavioral_agent.py*,

backend/app/agents/strategy_agent.py, and *backend/app/engine/simulation_loop.py*. These modules define agent behavior, market observations, decision loops, events, analytics, and portfolio effects. The project’s multi-agent nature is one of its strongest links to the literature.

Market microstructure literature maps to *backend/app/engine/order_book.py* and execution logic in *simulation_loop.py*. The platform’s support for price-time priority, partial fills, market orders, latency, slippage, market depth, and circuit breakers shows that execution is treated as a core system concern.

Reproducibility literature maps to *backend/app/core/research_store.py*, *backend/app/core/research_harness.py*, and *backend/app/api/research.py*.

These files implement the dataset, scenario, experiment, bot, run, evaluation, job, and event workflow. This is what turns StockAI from a simple simulation demo into a research platform.

Explainability and behavioral-finance literature map to the agent analytics and frontend dashboards. The generated report images in *generated/report_images/* also show that the project includes visual evidence of the research workspace, simulation dashboard, explainability panel, and live-market intelligence page.

Finally, full-stack engineering literature maps to the overall FastAPI, Pydantic, SQLite, static frontend, Chart.js, Pytest, and deployment structure. The project is not only an algorithm; it is an end-to-end application. This makes the literature survey broader but also makes the project stronger as a Bachelor of Engineering software and AI systems project.

2.22 Summary and Conclusion

The literature survey shows that StockAI v2.0 is best positioned at the intersection of financial AI, agent-based computational finance, market microstructure, behavioral simulation, explainable decision support, and research-platform engineering. Classical forecasting models such as ARIMA and GARCH provide useful baseline context, but they are not sufficient for modeling heterogeneous trader behavior or realistic execution. Machine learning and deep learning models offer stronger predictive flexibility, but they often remain disconnected from order-book mechanics and reproducible strategy evaluation. Reinforcement learning is highly relevant because trading is a sequential decision problem, but RL depends heavily on realistic simulation environments.

Agent-based computational finance is the closest academic foundation for StockAI v2.0. The

platform models a market as an interaction among different agents, events, sectors, regimes, and execution constraints. It supports deterministic strategies, behavioral agents, optional LLM-backed reasoning, and persistent research records. This allows users to study market behavior as a process rather than only as a prediction target.

Market microstructure literature further strengthens the motivation for the project. StockAI's order-book engine, latency, slippage, partial fills, market depth, and circuit-breaker behavior address weaknesses found in many prediction-only systems. Behavioral-finance literature supports the use of personas, biases, conviction, and decision logs. Explainable AI literature supports the need for decision transparency, especially in financial contexts.

The literature also identifies future opportunities. StockAI can be extended with historical-data ingestion, supervised ML baselines, LSTM or transformer forecasting modules, RL agents, stronger calibration, walk-forward evaluation, and richer transaction-cost modeling. These additions would build naturally on the platform's existing architecture.

In conclusion, StockAI v2.0 fills an important gap between simple stock dashboards and narrow prediction models. It provides a local-first, full-stack research platform where simulated markets, trading agents, execution logic, analytics, explainability, live-market comparison, and reproducible experiments are combined. From the literature perspective, its strongest contribution is not one isolated algorithm but the integration of multiple financial AI research concepts into a usable and extensible simulation environment.

3. EXISTING SYSTEM

3.1 Introduction

Currently, systems attempting to solve stock market analysis and simulation are fragmented into two distinct paradigms: purely predictive academic machine learning models and commercial algorithmic trading simulators. Academic models focus heavily on supervised learning pipelines, taking historical price and volume data to predict a future price point or trend. Conversely, commercial simulators provide realistic order execution and paper trading but require humans or hard-coded scripts to drive the decisions. Although both paradigms serve their specific niches, neither successfully merges autonomous AI agent behavior with persistent research orchestration and execution realism.

Existing predictive models typically ingest historical OHLCV (Open, High, Low, Close, Volume) data, apply statistical or deep learning transformations, and output a directional forecast. Commercial simulators, on the other hand, connect to live or delayed market data streams, allowing users to place simulated trades that are matched against a pseudo-order book. The intelligence in these systems is either entirely separated from the execution mechanics, or the execution mechanics are completely devoid of built-in artificial intelligence.

3.2 Limitations

3.2.1 Isolation from Market Microstructure

Most existing AI predictive models operate in a vacuum. They operate on the assumption that if a model predicts a price increase, the trade can be executed flawlessly at the currently observed price. They completely fail to account for market microstructure, such as the bid-ask spread, order book depth, execution latency, and price slippage that naturally occur when actual volume is traded in a live environment.

3.2.2 Lack of Autonomous Heterogeneous Agents

Traditional simulators test a single algorithmic strategy against historical data. They do not model the market as a living ecosystem of diverse, interacting participants. Existing systems lack

the architectural capability to simulate environments where Large Language Model-driven agents, rule-based retail bots, and institutional algorithmic strategies interact, compete, and influence the market state simultaneously.

3.2.3 The "Black Box" Explainability Problem

Most contemporary deep learning systems utilized in stock prediction, such as deep neural networks or complex ensemble methods, function as black boxes. They output basic buy, sell, or hold signals without providing actionable explainability. They cannot articulate the behavioral biases, conviction levels, or contextual reasoning behind a specific trade, making them exceptionally difficult to audit, tune, or trust in practical research scenarios.

3.2.4 Absence of End-to-End Research Reproducibility

Current platforms lack integrated, persistent state management for research orchestration. When analysts run financial simulations, the exact environment configurations, random seeds, agent definitions, and event tapes are rarely persisted in a structured, queryable database. This forces researchers to manually stitch together disparate datasets, model weights, and execution logs to recreate an experiment, severely hindering comparative analysis and scientific reproducibility.

3.3 Problem Statement

Despite the proliferation of financial forecasting tools, there remains a significant architectural challenge in developing platforms that seamlessly integrate autonomous artificial intelligence agents with realistic market execution mechanics. Existing systems typically segregate predictive modeling from order book simulation, resulting in fragmented environments where algorithmic strategies cannot be accurately tested against the emergent, unpredictable behaviors of heterogeneous market participants. The primary technical problem addressed by this system is the design and implementation of a cohesive, local-first research architecture capable of orchestrating a multi-agent synthetic stock market. This requires overcoming the computational complexities of synchronizing a continuous simulation kernel, maintaining an accurate price-time priority matching engine, and managing the asynchronous reasoning loops of Large Language Model-backed agents. Furthermore, the system must solve the data persistence problem by ensuring that all dynamic market states, agent decisions, and performance metrics are reliably recorded in a structured database to guarantee full experimental reproducibility and analytical explainability for financial AI research.

4. PROPOSED SYSTEM

4.1 Introduction

StockAI v2.0 is proposed as a comprehensive, local-first research platform meticulously designed to simulate a synthetic US equities market. It directly addresses the critical limitations of isolated predictive models and rigid commercial simulators by offering a hybrid architecture that seamlessly integrates autonomous artificial intelligence agents with highly realistic order execution mechanics. The system is engineered to orchestrate complex, multi-agent interactions within a controlled, observable environment, empowering researchers to define specific market scenarios, calibrate underlying datasets, and rigorously evaluate algorithmic strategies.

4.1.1 Key Repository-Derived Features

Feature	Evidence from Repository	Purpose
Landing page	frontend/landing.html	Product presentation
Research workspace	frontend/workspace.html	Dataset/scenario/bot/run orchestration
Simulator console	frontend/index.html	Live simulation, charts, agents, explainability
Live market page	frontend/live-market.html	Real-world market comparison
Backend APIs	backend/app/api/*	Serve market, simulation, agents, data, chat, research
Persistent research store	backend/app/core/research_store.py	SQLite-backed datasets, scenarios, runs, evaluations
Simulation kernel	backend/app/engine/simulation_loop.py	Market evolution and execution
Order book	backend/app/engine/order_book.py	Matching engine
Agent behaviors	backend/app/agents/*	LLM, rule-based, and strategy agents
Strategy SDK	backend/app/sdk/strategy.py	Deterministic research bot framework
Analytics	backend/app/core/analytics.py	Market and agent metrics
Evaluation harness	backend/app/core/research_harness.py	Multi-seed bot evaluation

Table 4.1 Key Functional Features Extracted from Repository

4.2 Advantages of the Proposed System

The proposed system offers several distinct advantages over conventional financial forecasting tools and standard trading simulators. Primarily, it replaces the traditional single-forecast output paradigm with a dynamic, multi-agent ecosystem, enabling the empirical study of emergent market behaviors driven by interacting participants with varying strategies and biases. Furthermore, the system introduces strict execution realism by natively incorporating vital market microstructure elements such as order latency, ensuring that algorithmic strategies are stress-tested under realistic trading constraints rather than idealized conditions. The platform also guarantees absolute experimental reproducibility and behavioral explainability by persisting all datasets, configuration scenarios, decision logs, and detailed run events within a structured local database. Additionally, the integration of a live-market intelligence layer allows researchers to directly contextualize and compare their synthetic simulation states against real-world market snapshots, significantly enhancing the practical validity of the research.

4.3 Specifications of the Proposed System

The architectural and functional specifications of StockAI v2.0 are defined by a modern, full-stack technology implementation paired with a robust simulation engine. The system's backend operates on FastAPI to provide high-performance, asynchronous RESTful endpoints, while state persistence is managed through a lightweight SQLite database utilizing Pydantic models for strict schema validation. The frontend specification requires an interactive, browser-based user interface constructed with HTML, CSS, and JavaScript, utilizing Chart.js for the real-time visualization of market analytics and agent performance. Functionally, the simulation kernel is specified to govern a 25-stock universe distributed across six major economic sectors, strictly enforcing price-time priority within its order book matching engine while dynamically processing correlated price walks, market events, session phases, and circuit breakers. Furthermore, the agent specification mandates support for the concurrent execution of Large Language Model-backed behavioural agents, deterministic Python strategy bots, and conditional rule-based actors. Finally, the system specifies comprehensive orchestration workflows, requiring the platform to reliably manage the creation, execution, and deterministic evaluation of multi-seed research runs while continuously logging actionable explainability data.

4.3.1 Proposed Architecture

The high-level system architecture of the StockAI simulation environment is structured into three highly cohesive operational layers. The foundational layer, designated as the Market Kernel, manages the raw ingestion of market data, including live quotes, order books, and historical fundamental data, routing it through an ingestion engine connected to persistent storage and external exchange APIs. The middle tier acts as the cognitive engine of the system, housing a sequential pipeline of Large Language Model (LLM) agents that process incoming market signals, execute complex decision-making logic, and interact with a centralized Analytics Store encompassing time-series databases and event logs. The top layer serves as the static frontend presentation tier, establishing a bidirectional flow of user interactions and visual data telemetry across dedicated Simulator, Workspace, and Live Monitor dashboards.

The platform also guarantees absolute experimental reproducibility and behavioural explainability by persisting all datasets, configuration scenarios, decision logs, and detailed run events within a structured local database. Additionally, the integration of a live-market intelligence layer allows researchers to directly contextualize and compare their synthetic simulation states against real-world market snapshots, significantly enhancing the practical validity of the research.

The refined architectural flow of StockAI v2.0 emphasizes local-first execution, robust data persistence, and structured multi-agent intelligence. At the foundational level, the Market Matching Kernel directly interfaces with live data feeds while enforcing strict price-time priority within its order matching engine, persisting all simulation states, agent parameters, and evaluation events locally within an SQLite database.

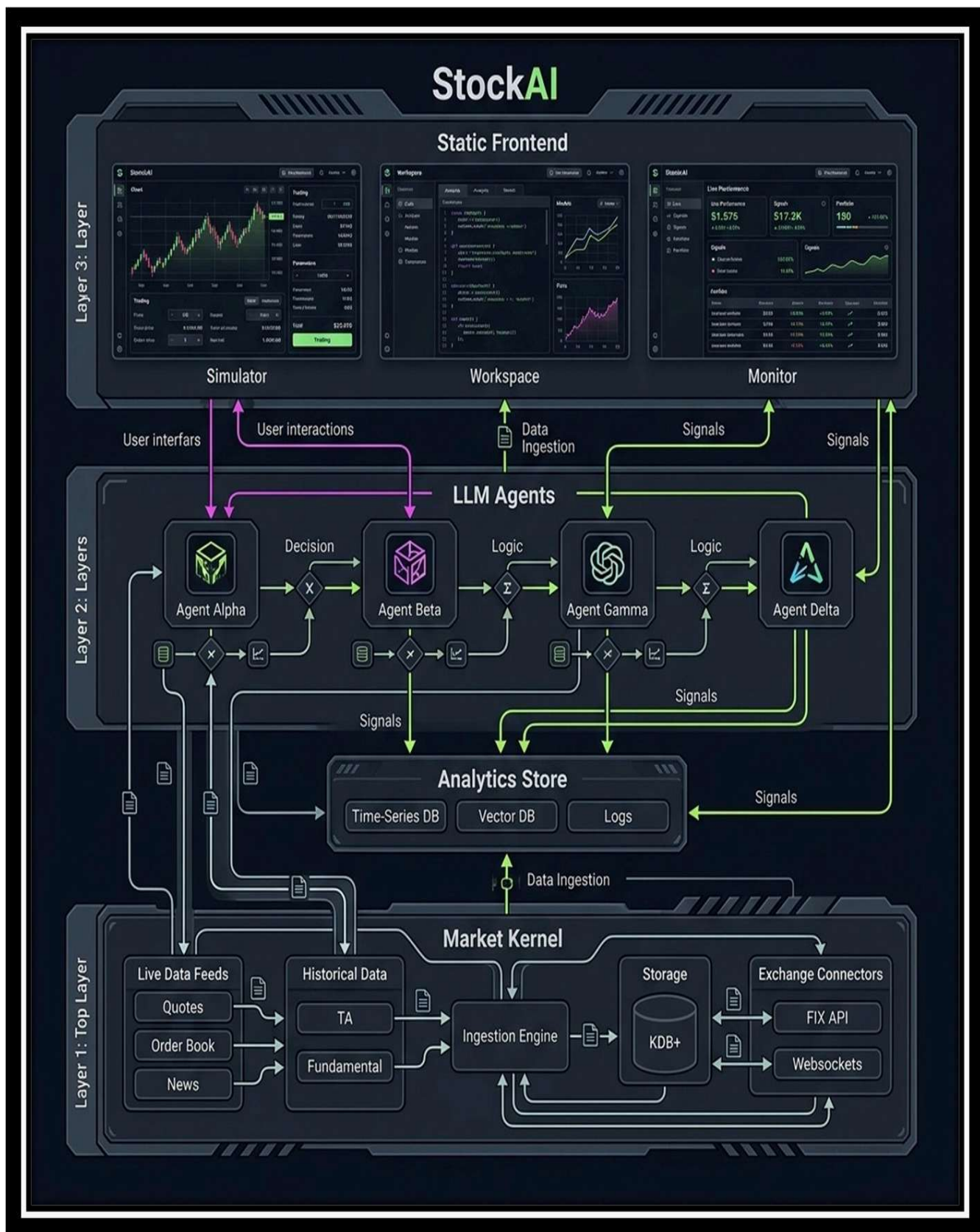


Fig. 4.1 System Architecture of the StockAI Framework

The intermediate Agent Intelligence Core orchestrates the behavioral logic of the simulation, translating inputs from neural LLM agents into defined trading personas and routing their actions through deterministic decision trees for execution. The entire ecosystem is orchestrated via the overarching Frontend Dashboards, which continuously poll the underlying layers to provide interactive visual interfaces for the Simulator, research Workspace, and Live Monitor telemetry.

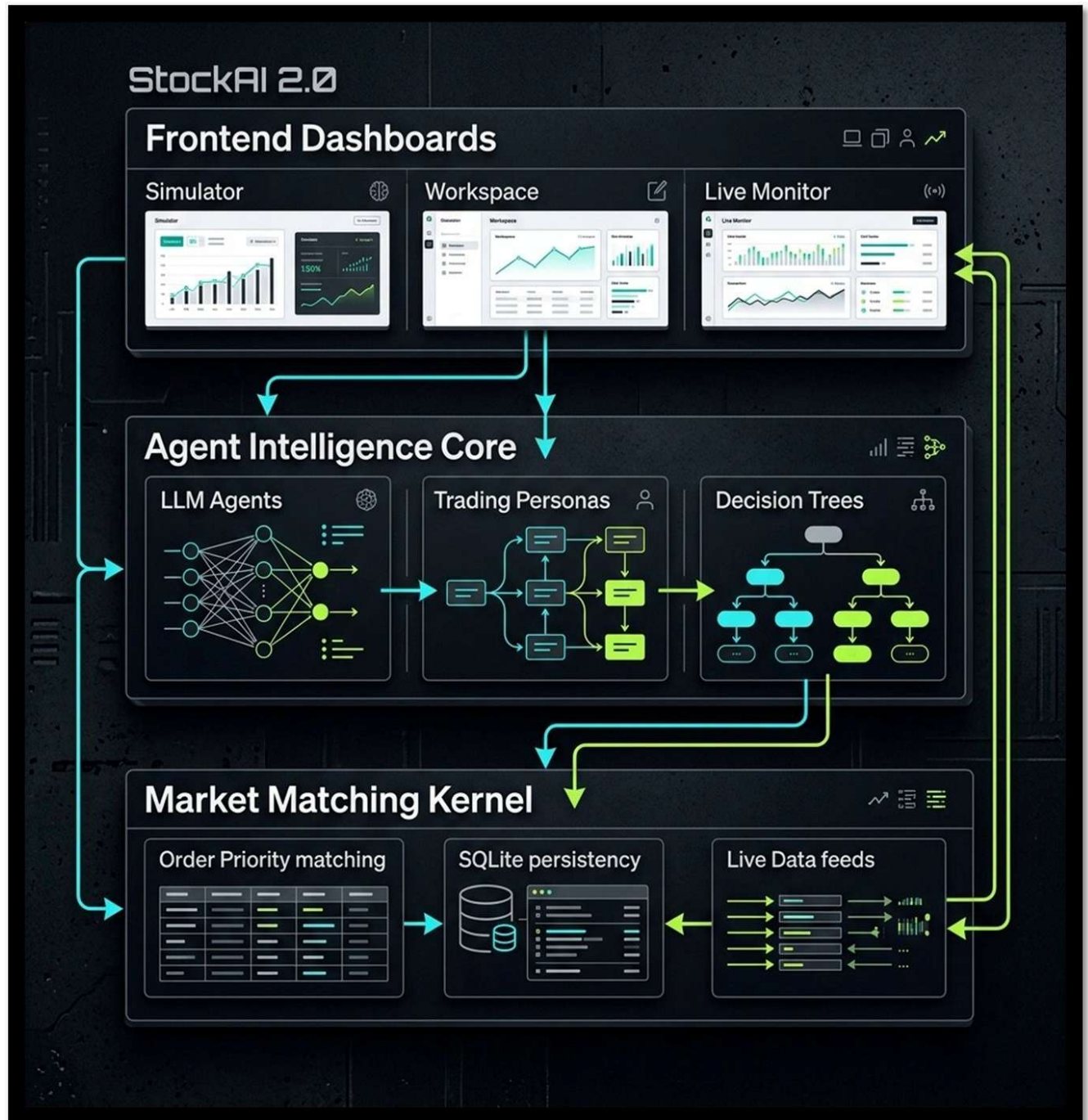


Fig.4.2 Refined Architectural Flow of StockAI v2.0

5. SYSTEM ANALYSIS

5.1 Introduction

System analysis is a critical phase in the development lifecycle that involves evaluating the technical, operational, and economic parameters required to successfully build and deploy the proposed platform. For StockAI v2.0, this phase analyzes the requirements for transitioning from conceptual financial modeling to a robust, interactive hybrid synthetic market laboratory. It meticulously defines the feasibility of the project, outlines the required system architecture, and establishes the strict functional and non-functional requirements necessary to orchestrate a multi-agent simulation kernel. Furthermore, it determines the precise hardware and software prerequisites needed to ensure seamless local-first execution, data persistence, and interactive user visualization.

5.2 Feasibility Study

A feasibility study objectively assesses the practicality and viability of the proposed system to ensure that the project can be successfully completed within the allocated constraints. This study evaluates technical capabilities, operational workflows, economic impacts, and legal compliance boundaries.

5.2.1 Technical Feasibility

The project is technically feasible because it uses stable and widely adopted technologies such as FastAPI and Uvicorn for backend service, SQLite for local persistence, Pydantic for schema validation, HTML/CSS/JavaScript for frontend delivery, Pytest for automated testing, and optional AI APIs through environment configuration. The repository already includes implementation artifacts for all major modules, confirming that the multi-agent trading architecture and continuous simulation loop are technically sound and fully capable of local execution.

5.2.2 Operational Feasibility

The system is highly operationally feasible because it can run locally from a single repository and provides intuitive, browser-based interfaces for orchestration and analysis. Furthermore, it supports deterministic operation when external AI keys are unavailable, ensuring continuous functionality offline, while exported research bundles enable comprehensive external reporting and downstream academic analysis.

5.2.3 Economical Feasibility

The project is economical for academic development because the core stack is open source, SQLite removes the need for a separate dedicated database server, the frontend is static and deployment-friendly, live market data utilizes a free Yahoo Finance chart endpoint, and the Large Language Model dependency is optional with a mock fallback configuration for local use. To formalize cost estimation, the Basic Constructive Cost Model (COCOMO) can be applied. Assuming StockAI v2.0 is an "organic" project developed by a small academic team in a familiar environment, and estimating the codebase at approximately KLOC (Kilo Lines of Code) for the backend, engine, and frontend integration, the effort is calculated using the formula

$$E = 2.4 * (KLOC)^{1.05}$$

This yields an estimated effort of roughly 12.6 person-months. Development time is calculated as

$$T = 2.5 * (E)^{0.38}$$

resulting in approximately 6.6 months of schedule time. Because this is developed in an academic setting utilizing free, open-source frameworks, the actual monetary cost is strictly limited to optional LLM API consumption and basic cloud hosting if deployed remotely, rendering the project highly economically feasible.

The system also improves economic value by supporting reusable simulation runs, datasets, scenarios, agents, evaluations, and exports. Researchers can compare multiple strategies without repeatedly rebuilding the environment. This reduces manual effort and makes the platform more cost-effective for experimentation.

The project is classified under the **Organic** category, as it is a relatively small, well-understood academic software project developed by a small team with flexible requirements.

For the Basic COCOMO Organic model, the standard constants are defined as:

- i. $a_b = 2.4$
- ii. $b_b = 1.05$
- iii. $c_b = 2.5$
- iv. $d_b = 0.38$

The estimated size of the source code is **21.5 KLOC** (*Kilo Lines of Code*).

a. Effort Estimation (E)

Effort is measured in Person-Months (PM) and represents the total amount of work required to complete the software.

$$E = a_b \times (KLOC)^{b_b}$$

$$E = 2.4 \times (21.5)^{1.05}$$

$$E = 2.4 \times 25.10$$

$$\mathbf{E = 60.24 \text{ Person-Months (approx.)}}$$

b. Development Time Estimation (D)

Development Time is measured in chronological months and represents how long the project will take to complete.

$$D = c_b \times (E)^{d_b}$$

$$D = 2.5 \times (60.24)^{0.38}$$

$$D = 2.5 \times 4.67$$

$$\mathbf{D = 11.68 \text{ Months (approx.)}}$$

c. Average Staffing (S)

Average Staffing represents the recommended number of personnel required to develop the system within the estimated development time.

$$S = E / D$$

$$S = 60.24 / 11.68$$

$$\mathbf{S = 5.16 \text{ Persons (approx.)}}$$

(This indicates a team size of approximately 5 to 6 developers would be ideal for commercial-scale execution).

d. Estimated Development Cost (C)

To estimate the commercial baseline cost of developing this platform, we apply a standard entry-level academic/research developer stipend of **₹20,000 per month**.

$$C = E \times \text{Monthly Salary}$$

$$C = 60.24 \times 20000$$

$$C = ₹ 12,04,800$$

Summary of Estimates

- i. **Total Source Code Size:** 21.5 KLOC
- ii. **Total Effort:** ~60.24 Person-Months
- iii. **Development Time:** ~11.68 Months
- iv. **Required Staff Size:** ~5 Developers
- v. **Total Estimated Cost:** ₹12,04,800

5.2.4 Legal Feasibility

The proposed system is completely legally feasible. The architecture strictly utilizes open-source software libraries and frameworks compliant with permissive licenses. Furthermore, the live market data integration securely queries public endpoints strictly for synthetic benchmarking and comparative intelligence, ensuring no proprietary financial data feeds are illegally redistributed. Because the platform operates purely as a synthetic simulation laboratory and executes no real monetary transactions, it entirely bypasses the strict regulatory compliance laws, licensing, and security mandates typically required by the Securities and Exchange Commission (SEC) or financial brokerage institutions.

5.3 System Implementation

The system implementation process translates the architectural blueprints into a functioning, local-first research platform. It is implemented using FastAPI to construct the backend, which manages asynchronous API routing, simulation orchestration, and active WebSocket connections for live UI updates. The core simulation kernel is implemented to iterate through defined session phases, generating event-driven price updates and strictly enforcing price-time priority logic within individual stock order books. Concurrently, the implementation connects Python-based strategy scripts, rule-based heuristics, and API-driven behavioral agents to observe these market states and asynchronously submit trading intents. Finally, the system implementation is solidified by SQLite, which ensures that all initialized datasets, agent logs, evaluated benchmarks, and active run events are serialized and persisted into a structured local database.

5.4 Functional Requirements

The functional requirements mandate that the system shall serve landing, simulator, workspace, and live-market pages while explicitly exposing stock metadata, live prices, history, and executing trades. Furthermore, the system shall provide the user with the ability to start, pause, stop, reset, and configure simulations, as well as seamlessly manage datasets, scenarios, experiments, bots, runs, evaluations, and background jobs. The engine is specifically required to simulate multi-agent decisions alongside realistic order execution, log sequential run events, allow for comprehensive run export, and accurately compute both market and agent-level analytics. Finally, the platform must support custom agent creation, provide textual explainability summaries for algorithmic actions, and fetch real-market snapshot data when network connectivity is available.

ID	Requirement
FR1	System shall serve landing, simulator, workspace, and live-market pages
FR2	System shall expose stock metadata, live prices, history, and trades
FR3	System shall start, pause, stop, reset, and configure simulations
FR4	System shall manage datasets, scenarios, experiments, bots, runs, evaluations, and jobs
FR5	System shall simulate agent decisions and order execution
FR6	System shall log run events and allow run export
FR7	System shall compute market and agent analytics
FR8	System shall support custom agent creation
FR9	System shall provide explainability summaries
FR10	System shall fetch real-market snapshots when available

Table 5.1 Functional Requirements

5.5 Non-Functional Requirements

The non-functional requirements dictate that the platform must deliver responsive API behavior optimized for local-first execution while maintaining strict reliability through deterministic fallback behaviors when live data or external LLM providers are suddenly unavailable. Maintainability is systematically ensured via a highly modular backend separated into distinct API,

engine, agents, models, and core layers, while software reusability is supported by a generalized strategy-SDK interface allowing for rapid new bot implementations. The architecture must also be highly scalable to support configurable agent counts and vast research entities, highly usable via static frontend pages featuring intuitive dashboard-like interactions, entirely portable for local execution via Python or Docker deployment, and strictly consistent through Pydantic validation for all structured inputs and outputs.

Category	Requirement
Performance	Responsive API behavior for local-first execution
Reliability	Fallback behavior when live data or LLM providers are unavailable
Maintainability	Modular backend with API, engine, agents, models, and core layers
Reusability	Strategy-SDK interface for new bot implementations
Scalability	Configurable agent counts and research entities
Usability	Static frontend pages with dashboard-like interaction
Portability	Local execution via Python, Docker, or Render deployment
Consistency	Pydantic validation for structured inputs and outputs

Table 5.2 Non-Functional Requirements

5.6 Hardware & Software Requirements

To guarantee optimal performance, the system requires a specific baseline of hardware and software configurations. The foundational software requirements for StockAI v2.0 include a Python 3.10 or higher runtime environment, SQLite for local database persistence, and a modern HTML5-compliant web browser to execute and render the interactive Chart.js frontend. Crucial Python library dependencies include FastAPI for the server framework, Uvicorn as the ASGI server, and Pydantic for rigid schema validation. Due to the platform's lightweight, local-first design, the hardware requirements remain highly accessible; a standard multi-core processor (such as an Intel Core i5 or equivalent) paired with a minimum of 8 GB of RAM is required to efficiently handle the concurrent execution of multiple AI agents alongside the high-frequency simulation loop. Additionally, at least 500 MB of free storage space is necessary to accommodate the repository codebase, the SQLite database growth, and exported JSON research bundles. Active internet connectivity is optional, required exclusively for querying external AI providers like OpenAI or Gemini, and for retrieving Yahoo Finance live-market intelligence payloads.

6. SYSTEM DESIGN

6.1 High-Level Architecture

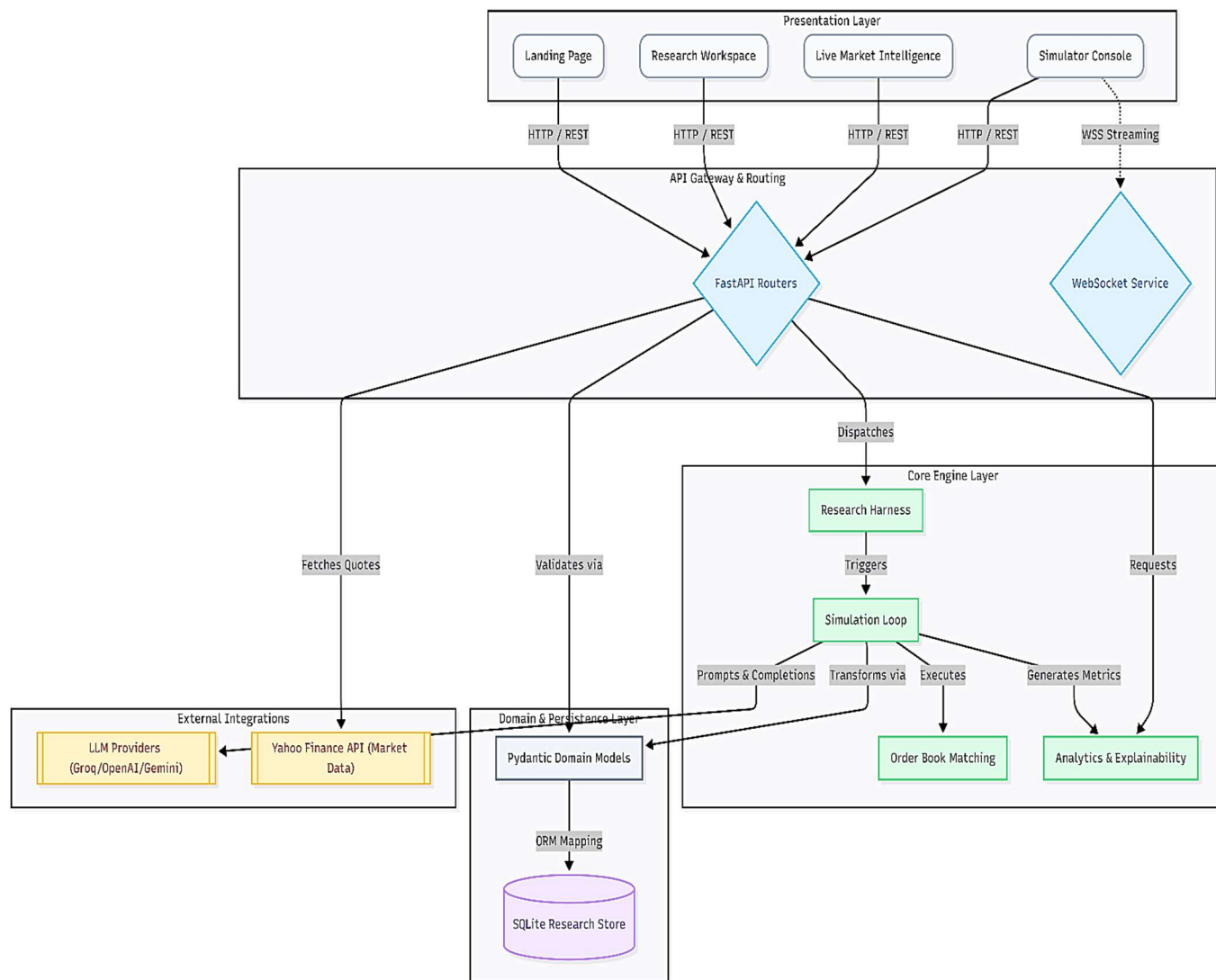


Fig: 6.1 Architecture Diagram

StockAI v2.0 follows a layered architecture:

- I. **Presentation Layer:** landing.html, workspace.html, index.html, live-market.html
- II. **API Layer:** market routes, simulation routes, agent routes, data routes, research routes, chat routes, live-market routes, websocket routes
- III. **Core and Engine Layer:** simulation loop, order book, analytics, research harness, job manager, live-market service

- IV. **Domain and Persistence Layer:** Pydantic models, SQLite research store, run events and evaluation records
- V. **External Integration Layer:** Groq/ Gemini providers, Yahoo Finance snapshot endpoint

6.2 All UML Diagrams

6.2.1 Data Flow Diagram Explanation

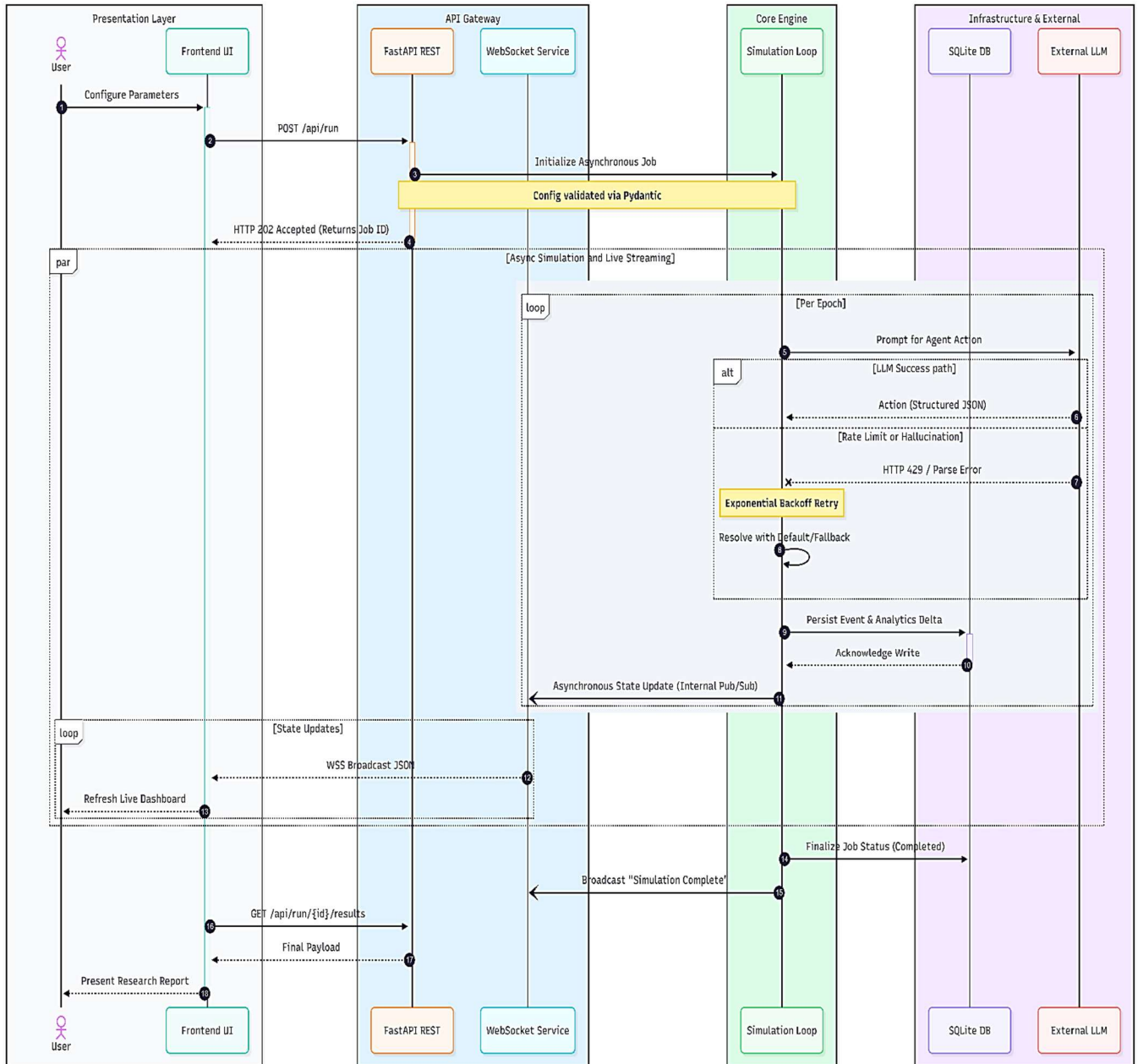


Fig: 6.2 Data Flow Diagram

The primary data flow is as follows:

- i. User configures the run through the frontend.
- ii. FastAPI receives the structured request.
- iii. SimulationConfig validates the input.
- iv. The state builder initializes the books, agents, and simulation world.
- v. The research store creates or updates the run metadata.
- vi. The simulation loop generates regimes, events, prices, and observations.
- vii. Agents produce their orders.
- viii. The order book matches the trades.
- ix. Analytics and run events are computed.
- x. The UI refreshes through API polling and websocket updates.
- xi. Results may be exported as research bundles.

6.2.2 Use Case Diagram

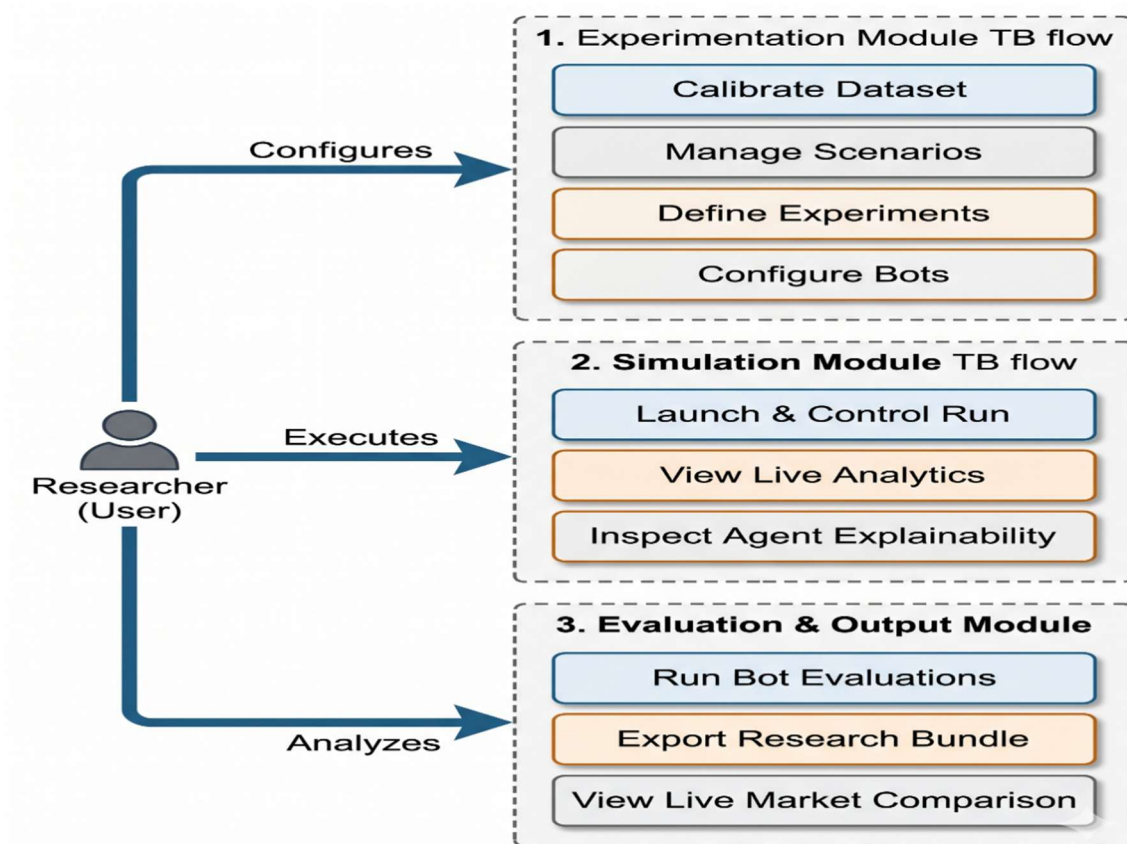


Fig: 6.3 Use Case Diagram

The primary actors for the system are researchers, students, and analysts. The main use cases include viewing the landing page, launching the simulator, calibrating the dataset, creating a scenario, creating an experiment, creating a bot, launching a run, starting, pausing, stopping, or resetting the simulation, inspecting market analytics, inspecting agent analytics, viewing explainability, running an evaluation, exporting a run bundle, and viewing live-market intelligence.

6.2.3 Class Diagram

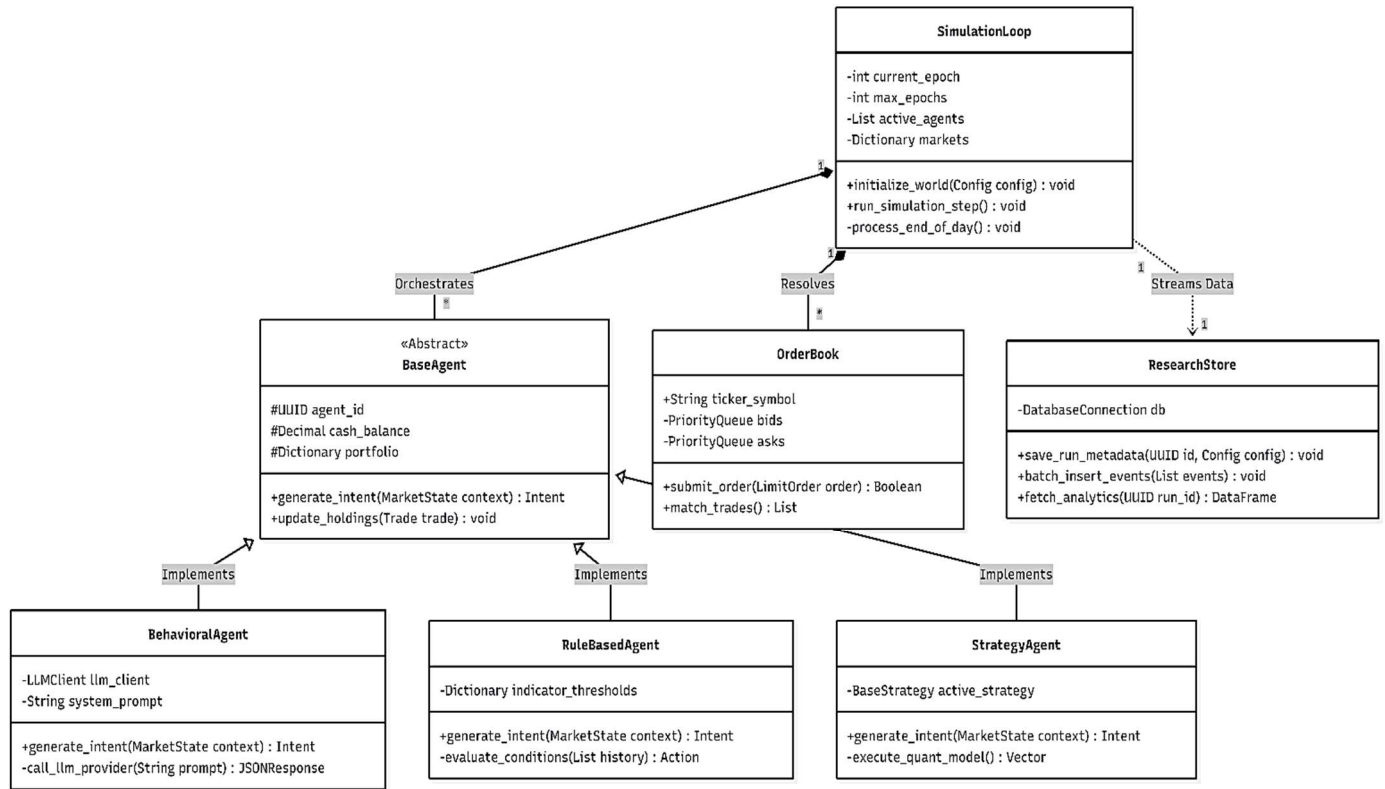


Fig: 6.4 Class Diagram

Important classes within the system include **SimulationLoop**, **OrderBook**, **BaseAgent**, **BehavioralAgent**, **RuleBasedAgent**, **StrategyAgent**, **ResearchStore**, **LiveMarketService**, **SimulationConfig**, **RunRecord**, and **EvaluationReportRecord**. The relationship summary indicates that the **SimulationLoop** contains agents and order books, while the **BehavioralAgent**, **RuleBasedAgent**, and **StrategyAgent** classes inherit from the **BaseAgent**. Additionally, the **StrategyAgent** composes a **BaseStrategy**, and the **ResearchStore** is responsible for persisting

datasets, scenarios, experiments, bots, runs, events, evaluations, and jobs.

6.2.4 Sequence Diagram

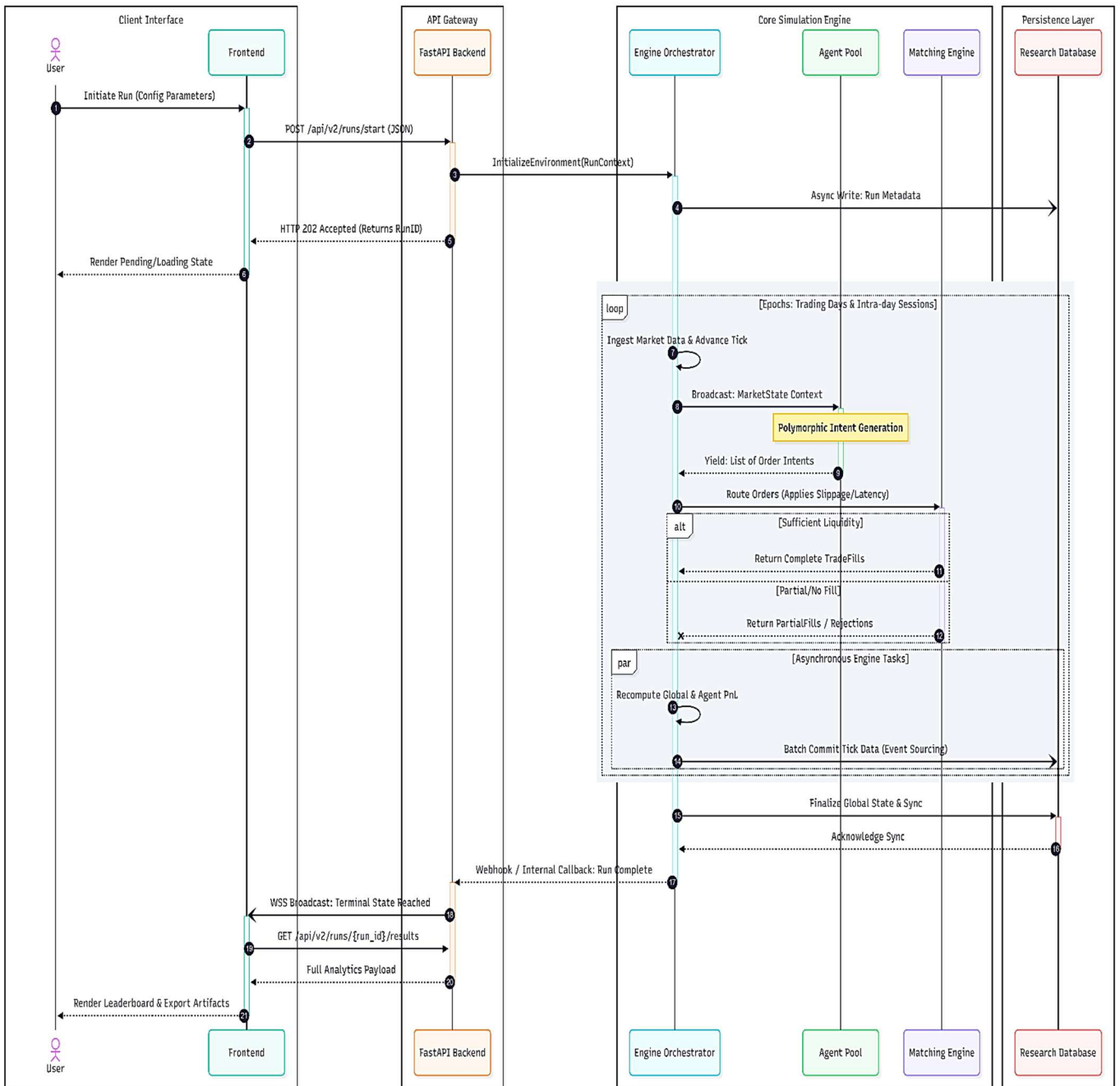


Fig: 6.5 Sequence Diagram

The sequence for the simulation run begins when the user clicks launch or start in the UI, prompting the frontend to call the /runs or /simulation/start endpoints. Next, the backend creates or activates the run, allowing the simulation loop to begin its day and session phases. As events

and price updates are generated, the agents observe the market state and submit their orders. The order book then matches these trades, after which the resulting analytics and events are stored in the database. Finally, the UI fetches the refreshed state to update the display for the user.

6.3 Database Design

The project uses SQLite as a lightweight local database stored under *backend/runtime/stockai_research.db*.

Entity	Purpose
datasets	calibrated market datasets
scenarios	regime and shock definitions
experiments	packaged research programs
agent_populations	population compositions
bots	strategy or deterministic bot definitions
runs	configured or executed research runs
run_events	ordered event tape per run
evaluations	benchmarking reports
jobs	background task tracking

Table 6.1 Core Entities in Research Database

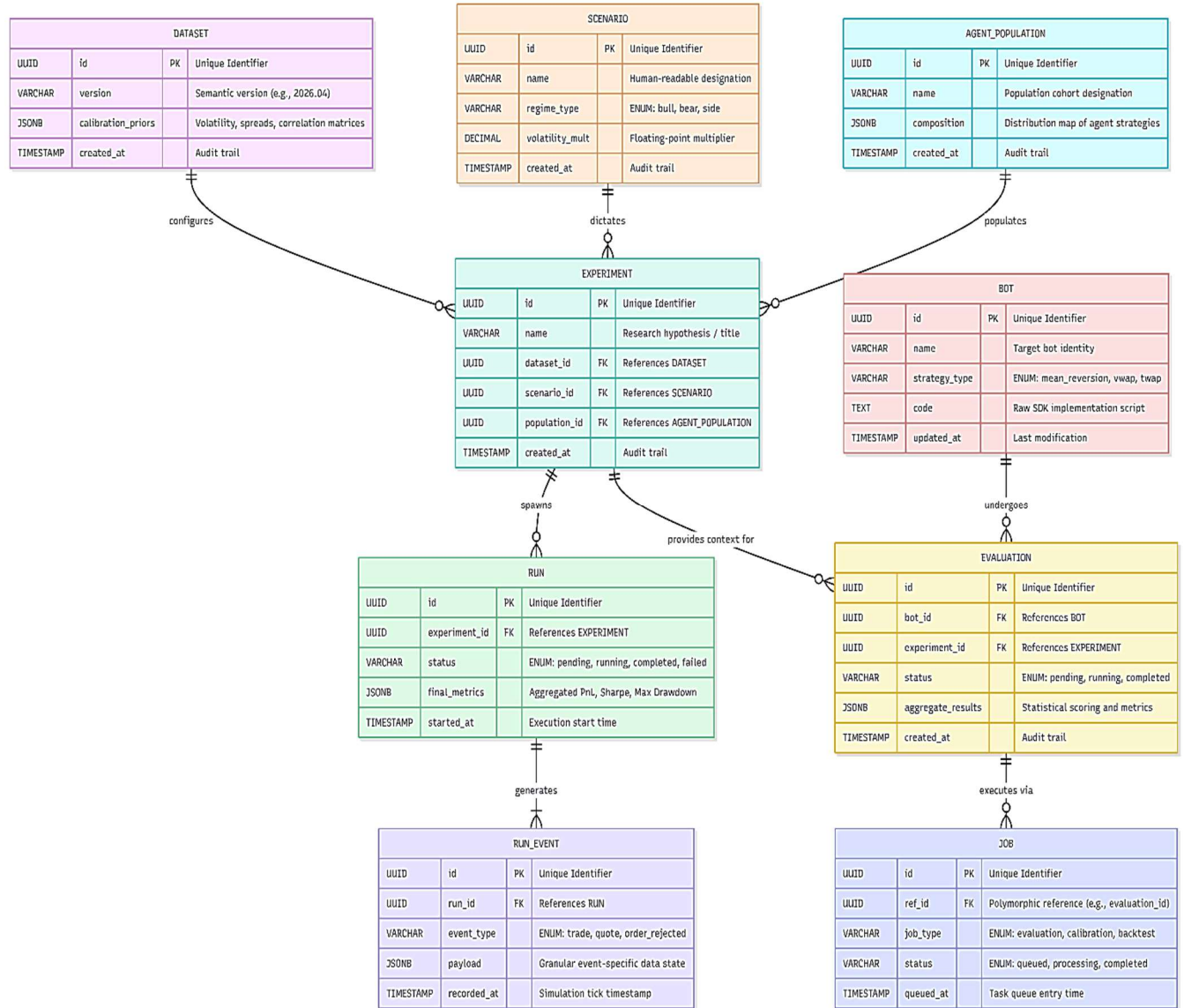


Fig: 6.6 Entity-Relational Diagram

Key relationships: one experiment references one scenario, one dataset, and one agent population, one run references one experiment and associated scenario/dataset/population, one run has many run events, one evaluation references one bot and optionally experiment/scenario/dataset context.

7. METHODOLOGY

7.1 Overall Pipeline

The implemented methodology is a hybrid simulation-and-evaluation workflow: data collection and universe seeding, preprocessing and configuration validation, calibration and feature derivation, agent population construction, simulation execution, analytics computation, and evaluation and export.

7.2 Step-by-Step Methodology

7.2.1 Data Collection

The project seeds a synthetic US-equities core universe with 25 stocks across Technology, Energy, Financials, Auto, Consumer, and Media. Each stock includes a symbol, company name, sector, initial price, volatility multiplier, benchmark, liquidity profile, market-cap bucket, beta, average daily volume, and description. Additional repository data comes from embedded calibration defaults in the research store, Excel artifacts in `res/`, and optional Yahoo Finance live snapshots.

7.2.2 Preprocessing

Preprocessing is handled through structured validation and state initialization, where `SimulationConfig` validates user parameters, the agent mix is normalized, and counts for LLM, rule, and strategy agents are derived. Furthermore, initial holdings are randomized, order books are created per stock, and dataset, scenario, and experiment references are explicitly attached.

7.2.3 Feature Engineering

The platform derives simulation features from multiple sources, including volatility bands, spread bands, liquidity profiles, sector correlations, event frequency, regime transition bias, benchmark index level, market breadth, realized volatility, turnover, sentiment, and session risk.

7.2.4 Model / Agent Selection

Instead of choosing a single predictive model, the system assembles a diverse agent population

consisting of a BehavioralAgent for LLM-backed reasoning, a RuleBasedAgent for deterministic threshold logic, and a StrategyAgent for SDK-based reproducible bots.

7.2.5 Training

Classical gradient-based model training is not the core mechanism here. Instead, the project uses deterministic benchmarking through multi-seed replay, evaluation harness execution, strategy callbacks and fill events, and persistent metrics storage.

7.2.6 Evaluation

Evaluation occurs through per-run analytics, agent metrics, explainability summaries, and aggregate bot evaluation metrics such as average PnL, average Sharpe ratio, average maximum drawdown, best seed, and worst seed.

7.2.7 Prediction / Decision Generation

Prediction in this repository is operationalized as agent action generation, which involves observing the market state, inferring or computing an action, creating an order, passing it through the execution kernel, and finally updating positions and metrics.

7.3 Pseudocode

Algorithm 7.3.1 World Initialization

Input: simulation_config **Output:** initialized world bundle 1. Validate simulation_config using SimulationConfig 2. Seed randomness if a seed is provided 3. Create one OrderBook per stock symbol 4. Set each order book last_price to stock.initial_price 5. Normalize agent mix into llm, rule, strategy ratios 6. Resolve integer counts for each agent type 7. Create behavioral agents 8. Create rule-based agents 9. Create strategy agents 10. Initialize SimulationLoop with agents and order books 11. Attach ResearchStore 12. Return world bundle

Algorithm 7.3.2 Simulation Execution

Input: active world bundle **Output:** updated market state and persisted run events 1. Mark simulation as running 2. For each step in total_days × sessions_per_day: 3. Compute current day and session phase 4. If new day: 5. roll regime 6. generate market events 7. process loans 8.

generate financial reports if needed 9. Derive sector impacts from events 10. Apply correlated price walk 11. Check circuit breakers 12. Build market_state observation 13. For each active agent: 14. generate order intent or hold 15. Queue orders according to latency 16. Match executable orders in order book 17. Apply slippage and update positions 18. Compute analytics and benchmark updates 19. Persist run event records 20. Broadcast tick update if websocket is active 21. End loop 22. Mark run completed or stopped

Algorithm 7.3.3 Strategy Evaluation

Input: bot_definition, seeds, num_days **Output:** aggregate evaluation report 1. For each seed: 2. Build deterministic configuration 3. Create strategy evaluator agent 4. Build world bundle with extra evaluator 5. Activate temporary evaluation run 6. Execute simulation for num_days 7. Collect final snapshot and analytics 8. Append seed-specific result 9. Compute average PnL, average Sharpe ratio, average drawdown 10. Identify best seed and worst seed 11. Save evaluation report 12. Return aggregate results.

8. DATASET DESCRIPTION

8.1 Dataset Source

The repository uses a combination of: an embedded default calibrated dataset named dataset-us-equities-core-v1, optional user-supplied calibration inputs via /datasets/calibrate, real-world live-market snapshots from Yahoo Finance for comparison, and spreadsheet artifacts stored in res/. The default dataset version stored in the research database is 2026.04.

8.2 Features

Per-stock seeded features include: symbol, name, sector, initial price, volatility multiplier, benchmark ETF, liquidity profile, market-cap bucket, beta, average daily volume in millions, and textual description. Calibration features include: low/medium/high/shock volatility bands, deep/core/satellite/thin spread bands, average daily volume profiles, sector correlation matrix, event frequency, and regime transition bias.

8.3 Data Size

Attribute	Value
Universe ID	us-equities-core-v1
Dataset ID	dataset-us-equities-core-v1
Stocks	25
Sectors	6
Benchmarks	QQQ, XLE, XLF, XLY, XLP, XLC, ICLN

Table 8.1 Seeded Market Universe Summary

File	Rows	Columns	Observed Purpose
stocks.xlsx	68	4	legacy/reference stock data
trades.xlsx	15	7	legacy trade records
agent_day_record.xlsx	1001	10	agent/day level records
agent_session_record.xlsx	3433	11	agent/session level records

Table 8.2 Repository Data Artifacts in res/

8.4 Train/Test Split

The repository does not implement a classic train/test split over historical stock data. Instead: calibration may be based on embedded priors or user-supplied reference values, simulation runs are controlled by seeds, and evaluation is conducted across multiple seeds and deterministic replay conditions. Therefore, the nearest equivalent to train/test methodology in this project is configuration and calibration stage as environment setup, and multi-seed evaluation stage as repeatable benchmarking. This distinction should be stated explicitly during academic presentation to avoid misclassifying the project as a standard supervised training pipeline.

9. MODULE IMPLEMENTATION

9.1 Data Processing Module

This module manages configuration validation, dataset calibration, persistence, and structured research records. **Main Files:** backend/app/models/types.py, backend/app/models/research.py, backend/app/core/research_store.py, backend/app/core/research_harness.py.

Key Logic: Pydantic models validate API payloads and domain entities, ResearchStore initializes SQLite schema and seeds defaults, Calibration pipeline converts user reference inputs into bounded market priors, and Records are serialized as JSON into SQLite tables.

9.2 Model Training / Evaluation Module

Although classical ML training is absent, this module performs deterministic bot evaluation and benchmarking.

Main Files: backend/app/core/research_harness.py, backend/app/agents/strategy_agent.py, backend/app/sdk/strategy.py.

Key Logic: builds seeded deterministic worlds, injects evaluator strategy agents, runs short simulations, collects PnL, Sharpe ratio, win rate, and drawdown, and aggregates metrics into evaluation reports.

9.3 Prediction / Simulation Module

This is the central intelligence module that evolves the market and converts agent decisions into executed trades.

Main Files: backend/app/engine/simulation_loop.py, backend/app/engine/order_book.py, backend/app/agents/behavioral_agent.py, backend/app/core/analytics.py.

Key Logic: regime rolling and event generation, sector-correlated price updates, market_state construction, LLM/rule/strategy action generation, latency queueing and slippage application, price-time-priority matching, and analytics and run-event persistence.

9.4 UI / API Module

This module presents user-facing surfaces and exposes backend services. **Main Files:** backend/app/main.py, backend/app/api/*.py, frontend/landing.html, frontend/workspace.html, frontend/index.html, frontend/live-market.html. **Key Logic:** serves frontend pages directly through FastAPI, exposes REST endpoints for all major functionalities, supports live dashboard interaction, provides run export and research workspace actions, and compares real-market.

9.5 Code Snippets

9.5.1. Server and API Layer

1. FastAPI Application Setup

```
# backend/app/main.py
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import FileResponse
from pathlib import Path
from datetime import datetime
from backend.app.api import market, simulation, agents, chat, data, ws, live_market, research

app = FastAPI(title="StockAI v2.0")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(market.router)
app.include_router(simulation.router)
app.include_router(agents.router)
app.include_router(chat.router)
app.include_router(data.router)
app.include_router(ws.router)
app.include_router(live_market.router)
app.include_router(research.router)

@app.get("/health")
async def health():
    return {"status": "StockAI v2.0 Online", "time": str(datetime.now())}
```

2. Simulation Status API

```
# backend/app/api/simulation.py
@router.get("/status")
async def get_status():
    sim = state.simulation
    prices = {
        s: state.market_books[s].last_price or state.STOCKS[s].initial_price
        for s in state.STOCKS
    }
```

```

analytics = compute_market_analytics(sim, prices, state.STOCKS)
return {
    "is_running": sim.is_running,
    "is_paused": sim.is_paused,
    "day": sim.day,
    "session": sim.session,
    "total_trades": sim.total_trade_count,
    "active_agents": sum(1 for a in state.agents if a.status == "active"),
    "stocks": {
        s: {"name": state.STOCKS[s].name, "price": prices[s]}
        for s in state.STOCKS
    },
    "market_analytics": analytics,
}

```

9.5.2. Core Engine and Microstructure

1. Main Simulation Loop

```

# backend/app/engine/simulation_loop.py
async def run_simulation(self, steps: Optional[int] = None):
    self.is_running = True
    total_steps = steps or self.total_days * self.sessions_per_day

    for step_index in range(total_steps):
        if not self.is_running:
            break

        while self.is_paused:
            await asyncio.sleep(0.5)

        self.day = (step_index // self.sessions_per_day) + 1
        self.session = (step_index % self.sessions_per_day) + 1
        self.session_phase, clock_time = self._phase_clock(self.session)

        if self.session == 1:
            self._roll_regime(self.day)
            day_events = self._generate_events(self.day)
            self.events.extend(day_events)

        self._apply_correlated_walk(self._generate_sector_drifts({}))

        prices = {
            s: book.last_price or self.base_prices.get(s, 100.0)
            for s, book in self.order_books.items()
        }

        market_state = {
            "day": self.day,
            "session": self.session,
            "session_phase": self.session_phase,
            "prices": prices,
            "regime": self.current_regime,
            "timestamp": utcnow(),
        }
        await self.step(market_state, "Market session update")
        await asyncio.sleep(max(self.speed, 0.0))

    self.is_running = False

```


2. Order Book Matching Logic

```
# backend/app/engine/order_book.py
class OrderBook:
    def add_order(self, order: Order) -> List[Trade]:
        if order.side == OrderSide.BUY:
            new_trades = self._match_buy_order(order)
        else:
            new_trades = self._match_sell_order(order)

        self.trades.extend(new_trades)
        return new_trades

    def _execute_trade(self, taker: Order, maker: Order, quantity: int, price: float,
trade_list: List[Trade]):
        taker.filled_quantity += quantity
        maker.filled_quantity += quantity
        self.last_price = price

        taker.status = OrderStatus.FILLED if taker.filled_quantity >= taker.quantity else
OrderStatus.PARTIALLY_FILLED
        maker.status = OrderStatus.FILLED if maker.filled_quantity >= maker.quantity else
OrderStatus.PARTIALLY_FILLED

        trade = Trade(
            trade_id=f"t_{int(time.time()*1000000)}_{uuid.uuid4().hex[:6]}",
            buy_order_id=taker.id if taker.side == OrderSide.BUY else maker.id,
            sell_order_id=taker.id if taker.side == OrderSide.SELL else maker.id,
            buyer_agent_id=taker.agent_id if taker.side == OrderSide.BUY else maker.agent_id,
            seller_agent_id=taker.agent_id if taker.side == OrderSide.SELL else
maker.agent_id,
            stock_symbol=self.symbol,
            price=price,
            quantity=quantity,
        )
        trade_list.append(trade)
```

9.5.3. Artificial Intelligence Integration

Agent Decision Making With LLM Fallback

```
# backend/app/agents/behavioral_agent.py
async def act(self, market_state: Dict[str, Any], news: str) -> Optional[Order]:
    if self.status == "bankrupt":
        return None

    if not self.llm:
        return self._demo_act(market_state)

    self._update_pnl(market_state["prices"])
    biases = self._determine_biases(market_state)

    prompt = PromptFactory.create_trade_prompt(
        agent_profile={**self.persona, "wallet": self.wallet, "pnl": self.pnl},
        market_state=market_state,
        active_biases=biases,
        news=news,
        available_stocks=[
            s for s in market_state["prices"]
            if s not in market_state.get("halted", set())
        ]
    )
```

```

    ],
)

response_str = await asyncio.to_thread(
    self.llm.generate,
    prompt=prompt,
    system_message="You are a realistic trading agent. Output JSON only.",
)

data = json.loads(response_str)
action = data.get("action", "hold").lower()

if action == "hold":
    return None

return Order(
    id=str(uuid.uuid4()),
    agent_id=str(self.id),
    stock_symbol=data["stock"],
    side=OrderSide.BUY if action == "buy" else OrderSide.SELL,
    type=OrderType.LIMIT,
    price=float(data["price"]),
    quantity=int(data["quantity"]),
    timestamp=market_state.get("timestamp"),
)

```

9.5.4. Real-Time Communication & External Data

1. WebSocket Live Updates.

```

# backend/app/api/ws.py
_clients: Set[WebSocket] = set()

async def broadcast(data: dict):
    if not _clients:
        return
    payload = json.dumps(data, default=str)
    disconnected = set()
    for ws in list(_clients):
        try:
            await ws.send_text(payload)
        except Exception:
            disconnected.add(ws)

    _clients -= disconnected

@router.websocket("/ws")
async def websocket_endpoint(ws: WebSocket):
    await ws.accept()
    _clients.add(ws)

    state.simulation.ws_broadcast = broadcast

    try:
        while True:
            data = await ws.receive_text()
            if data == "ping":
                await ws.send_text(json.dumps({"type": "pong"}))
    except WebSocketDisconnect:
        _clients.discard(ws)

```

2. Live Market Snapshot API

```
# backend/app/api/live_market.py
from fastapi import APIRouter, Query
from backend.app.core.live_market import live_market_service

router = APIRouter(prefix="/api/live-market", tags=["live-market"])

@router.get("/snapshot")
async def get_live_market_snapshot(
    refresh: bool = Query(False, description="Force a provider refresh")
):
    return await live_market_service.get_snapshot(force_refresh=refresh)
```

9.5.5. Frontend and User Interface

Frontend API Integration

```
// frontend/index.html
async function apiFetch(path, opts) {
    const response = await fetch(API + path, opts);

    if (!response.ok) {
        const text = await response.text();
        throw new Error(text || path);
    }

    return response.json();
}

async function refreshSimulationStatus() {
    const status = await apiFetch('/simulation/status');

    renderMarketCards(status.stocks);
    renderAnalytics(status.market_analytics);
    updateSimulationControls(status);
}
```

10. TESTING

This section lists concrete repository-grounded test cases. It avoids generic testing theory and focuses on actual observable inputs and outputs. Testing note: direct pytest collection failed in the analysis environment because the package groq was not installed and is imported at module load time. To obtain actual API outputs without modifying the repository, representative endpoints were exercised using an in-process stub of the missing module.

Test Case ID	Input	Expected Output	Actual Output
TC1	GET /health	API should return online status JSON	Status 200; returned {"status":"StockAI v2.0 Online", ...}
TC2	GET /market/stocks	All seeded stocks with metadata and live price	Status 200; returned 25 stocks including AAPL, MSFT, NVDA, TSLA, etc.
TC3	GET /simulation/status	Simulation metadata and analytics	Status 200; returned run_id, dataset_version, scenario_id, training_mode, latency_ms, slippage_bps, benchmark and breadth fields
TC4	GET /workspace/summary	Research registry summary	Status 200; counts showed 1 dataset, 1 scenario, 1 experiment, 2 bots, 1 population; active run configured
TC5	POST /datasets/calibrate with returns [0.01,0.015,0.02,0.03], spreads [3.5,5,9,12], volumes [12,15,22,28]	Dataset calibration should update with bounded values	Status 200; calibration updated to low=0.01, medium=0.0187, high=0.03, shock=0.0495, spreads and volumes recomputed
TC6	POST /bots with strategy mean_reversion	New strategy bot definition should be created	Status 200; bot created with id like bot-a3713207 and strategy mean_reversion

TC7	POST /evaluations using created bot, seeds [5,7], num_days=1, run_async=false	Evaluation report should complete and aggregate metrics	Status 200; evaluation completed with metrics avg_pnl=0.0, avg_sharpe_ratio=0.0, avg_max_drawdown=0.0, best_seed=5, worst_seed=5
TC8	OrderBook price-time priority test from repository	Earlier higher bid should fill before later equal-price bid	Verified in tests/test_research_platform.py logic; matching order sequence respects price then time
TC9	Market order non-resting behavior test from repository	Market order should fill against resting sell order and not rest on book	Verified in repository test logic using OrderBook("AAPL")
TC10	Workspace HTML route test	/workspace should contain orchestration controls	Repository test checks for launchRunBtn, createBotBtn, runEvalBtn, calibrateBtn, exportRunBtn

Table 10.1 Actual Test Cases and Observed Outputs

10.1 Additional Test Observations from Repository

The repository test suite also checks: /market/{symbol} returns symbol details, unknown symbols return 404, /simulation/config rejects changes when the simulation is running, speed validation enforces bounds, /agents/{id}/analytics exposes extended risk metrics, /api/live-market/snapshot supports both live and fallback payload shapes, /runs/{run_id}/events exposes run-scoped event streams, /runs/{run_id}/export returns bundle data, and experiment creation validates foreign references.

10.2 Testing Pytest Snippets

10.2.1. Health and Frontend Tests

```
# tests/test_critical_paths.py
from fastapi.testclient import TestClient
from backend.app.main import app

client = TestClient(app)
```

```

class TestHealth:
    def test_health_endpoint(self):
        response = client.get("/health")

        assert response.status_code == 200
        assert response.json()["status"] == "StockAI v2.0 Online"

    def test_frontend_served(self):
        response = client.get("/app")

        assert response.status_code == 200
        assert "text/html" in response.headers["content-type"]

```

10.2.2. Simulation API Tests

```

# tests/test_critical_paths.py
class TestSimulation:
    def test_simulation_status_shape(self):
        response = client.get("/simulation/status")

        assert response.status_code == 200
        data = response.json()

        for key in (
            "is_running",
            "is_paused",
            "day",
            "total_days",
            "total_trades",
            "active_agents",
        ):
            assert key in data

    def test_config_validation_speed_bounds(self):
        response = client.post(
            "/simulation/config",
            json={"num_days": 1, "speed": 0.0},
        )

        assert response.status_code == 422

```

10.2.3. Agent API Tests

```

# tests/test_critical_paths.py
class TestAgents:
    def test_list_agents(self):
        response = client.get("/agents")

        assert response.status_code == 200
        agents = response.json()

        assert len(agents) > 0
        for agent in agents:
            for key in ("id", "name", "cash", "pnl", "status", "trades"):
                assert key in agent

    def test_agent_analytics(self):
        agents = client.get("/agents").json()
        agent_id = agents[0]["id"]

```

```

response = client.get(f"/agents/{agent_id}/analytics")

assert response.status_code == 200
data = response.json()

for key in ("sharpe_ratio", "max_drawdown", "win_rate", "total_trades"):
    assert key in data

```

10.2.4. Execution Kernel Test

```

# tests/test_research_platform.py
from datetime import datetime, timedelta

from backend.app.engine.order_book import OrderBook
from backend.app.models.types import Order, OrderSide, OrderType

class TestExecutionKernel:
    def test_market_order_matches_and_does_not_rest(self):
        book = OrderBook("AAPL")

        sell_order = Order(
            id="sell-1",
            agent_id="seller-a",
            stock_symbol="AAPL",
            side=OrderSide.SELL,
            type=OrderType.LIMIT,
            price=100.0,
            quantity=5,
            timestamp=datetime.now(),
        )

        book.add_order(sell_order)

        market_buy = Order(
            id="buy-1",
            agent_id="buyer-a",
            stock_symbol="AAPL",
            side=OrderSide.BUY,
            type=OrderType.MARKET,
            price=0.0,
            quantity=3,
            timestamp=datetime.now() + timedelta(milliseconds=1),
        )

        trades = book.add_order(market_buy)

        assert len(trades) == 1
        assert trades[0].sell_order_id == "sell-1"
        assert market_buy.filled_quantity == 3
        assert len(book.bids) == 0

```

10.2.5. Research Workspace Tests

```

# tests/test_research_platform.py
class TestResearchWorkspace:
    def test_workspace_page_served(self):
        response = client.get("/workspace")

        assert response.status_code == 200
        assert "text/html" in response.headers["content-type"]

```

```

    html = response.text
    for token in (
        "launchRunBtn",
        "createBotBtn",
        "runEvalBtn",
        "calibrateBtn",
        "exportRunBtn",
    ):
        assert token in html

def test_research_registry_routes(self):
    for path in (
        "/datasets",
        "/scenarios",
        "/experiments",
        "/bots",
        "/agent-populations",
        "/runs",
        "/jobs",
    ):
        response = client.get(path)

        assert response.status_code == 200
        assert isinstance(response.json(), list)

```


11. SCREENSHOTS

The project contains four main visual surfaces. The screenshots from the running application at the following positions are displayed and exhibited.

11.1 Landing Page

The landing page presents StockAI v2.0 as an AI-powered market simulator with sections for hero overview, feature cards, ticker strip, live-market introduction, statistics, and call-to-action buttons for simulator, workspace, and live-market pages.

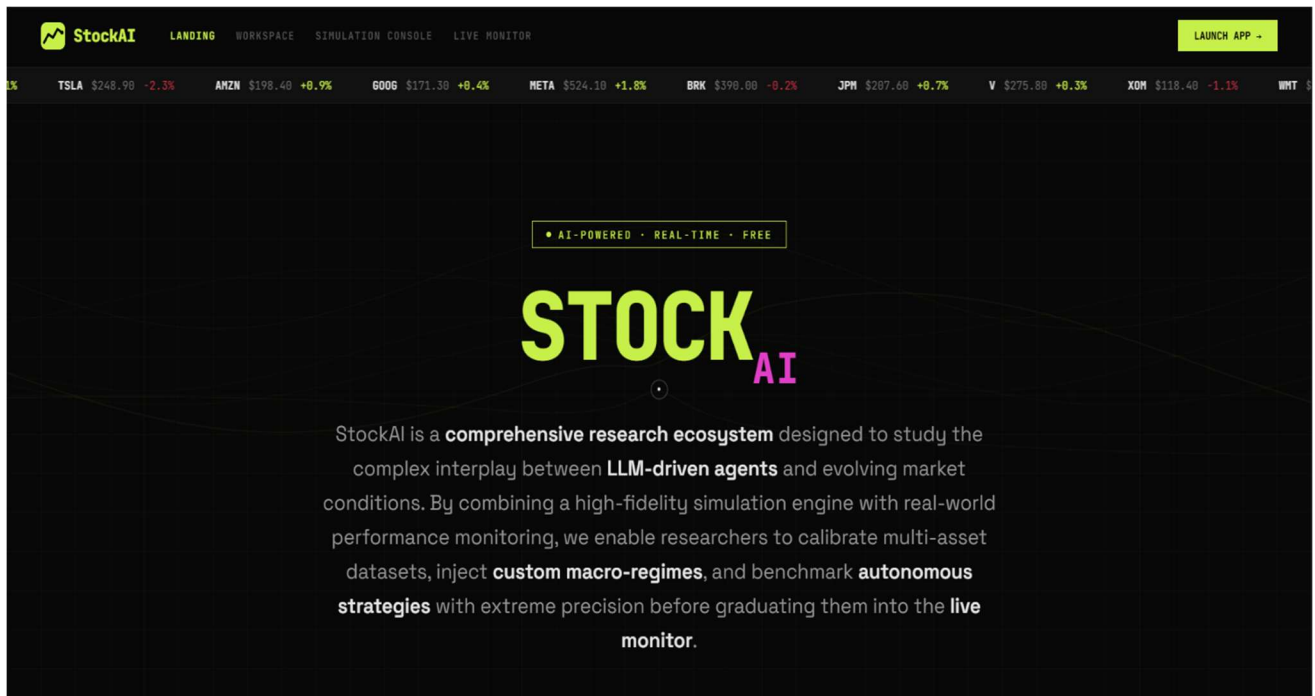


Fig 11.1: Screenshot of Home Page

11.2 Research Workspace

The workspace page acts as the orchestration layer. It includes: research pulse, next-best actions, control deck, active run metadata, kernel KPIs, run event tape, dataset/scenario/experiment registries, bot and population lists, and evaluation leaderboard and jobs.

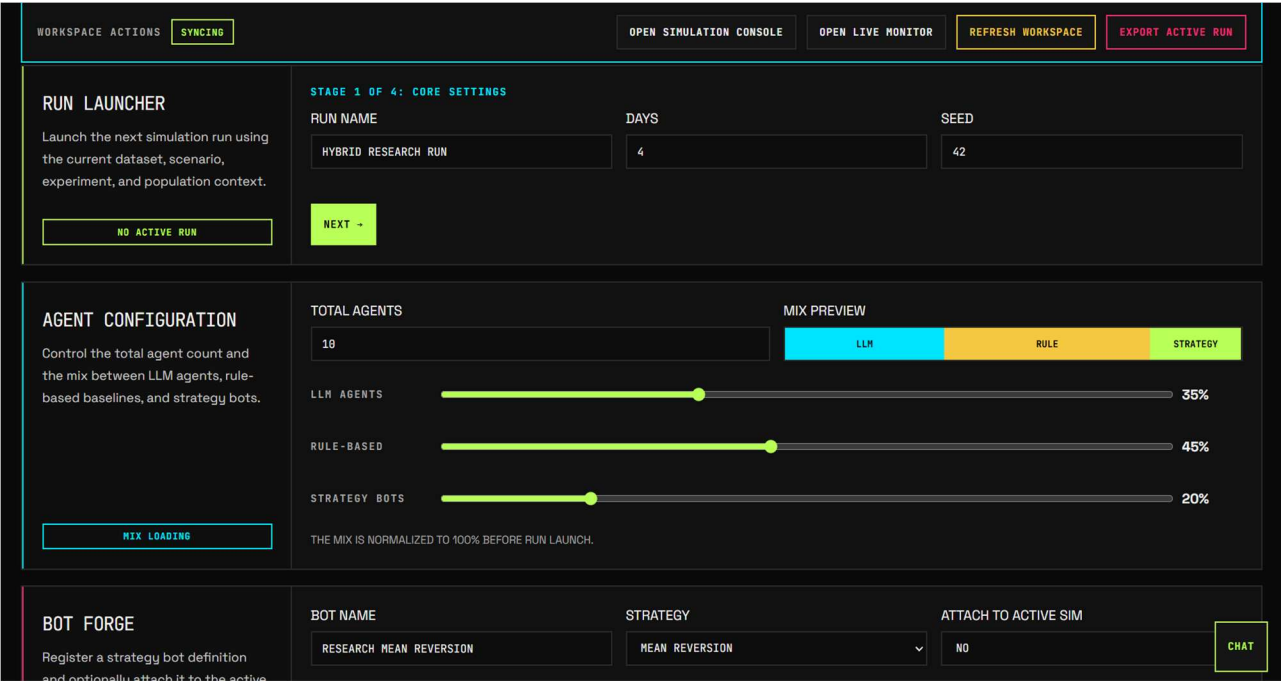
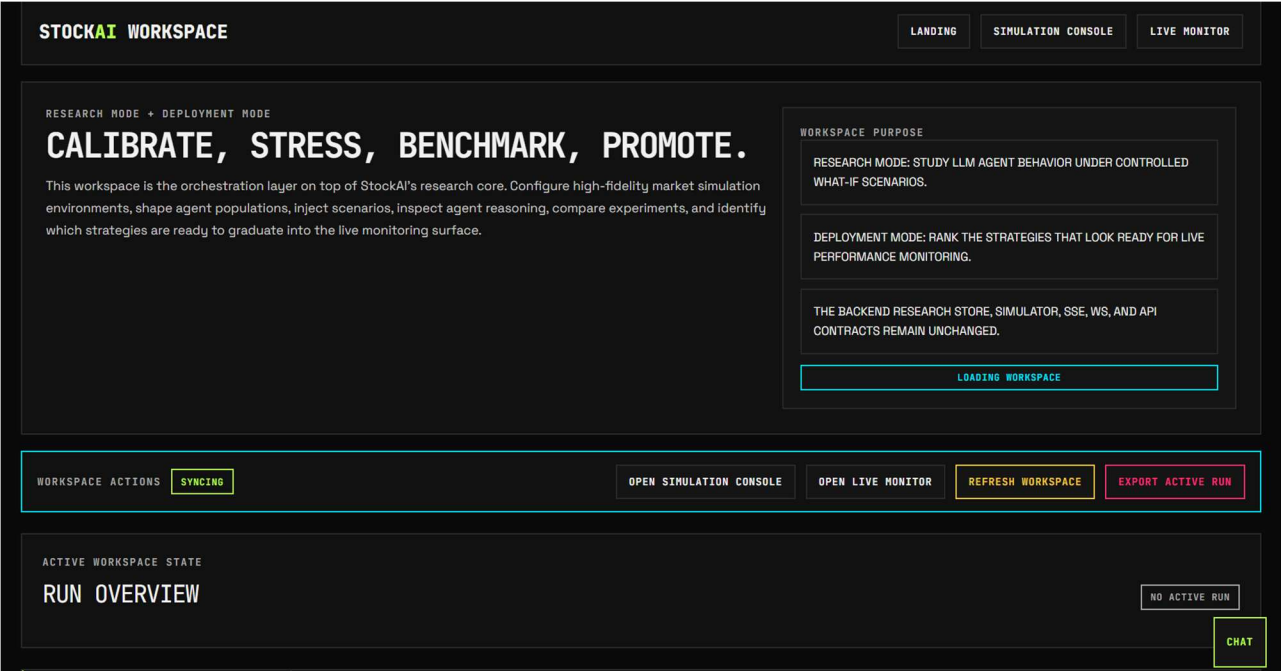


Fig 11.2: Screenshots of Research Workspace

11.3 Simulation Dashboard

The simulator UI is the most feature-rich page. Repository analysis shows sections for: dashboard, analysis, agents, explainability, settings, live price chart, heatmap, chatbot orb, price alerts, agent comparison, and onboarding tour.

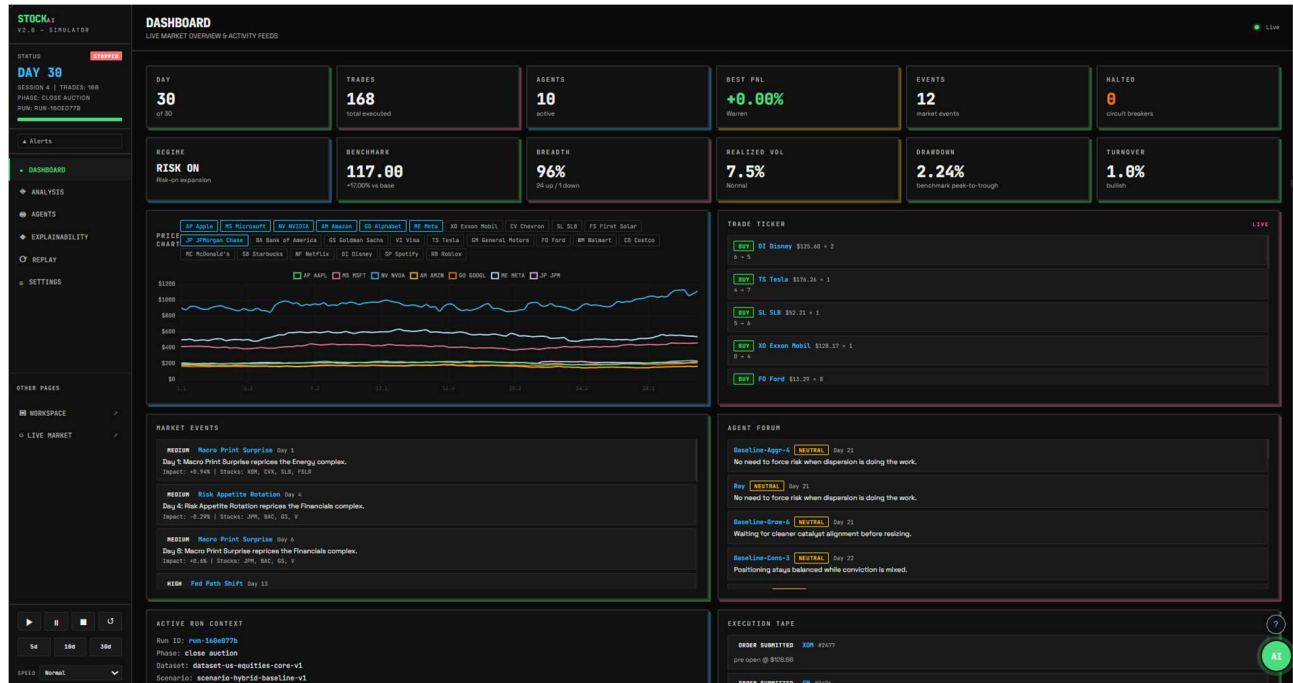
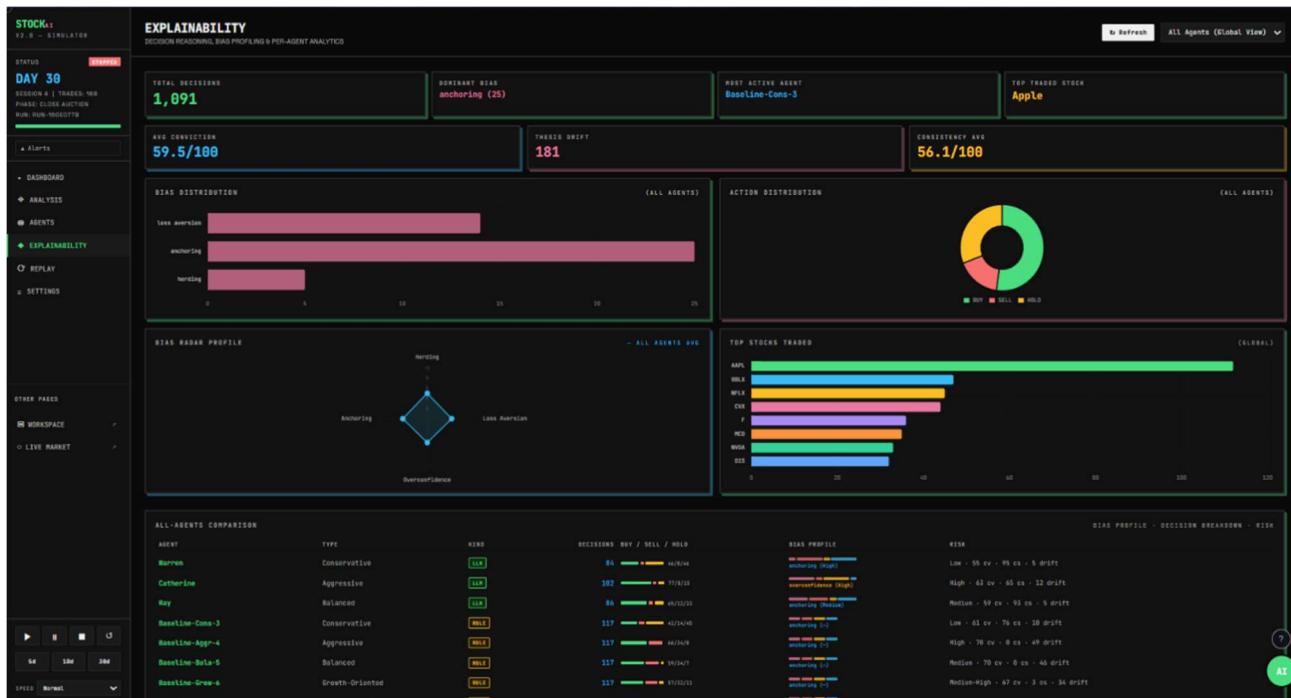
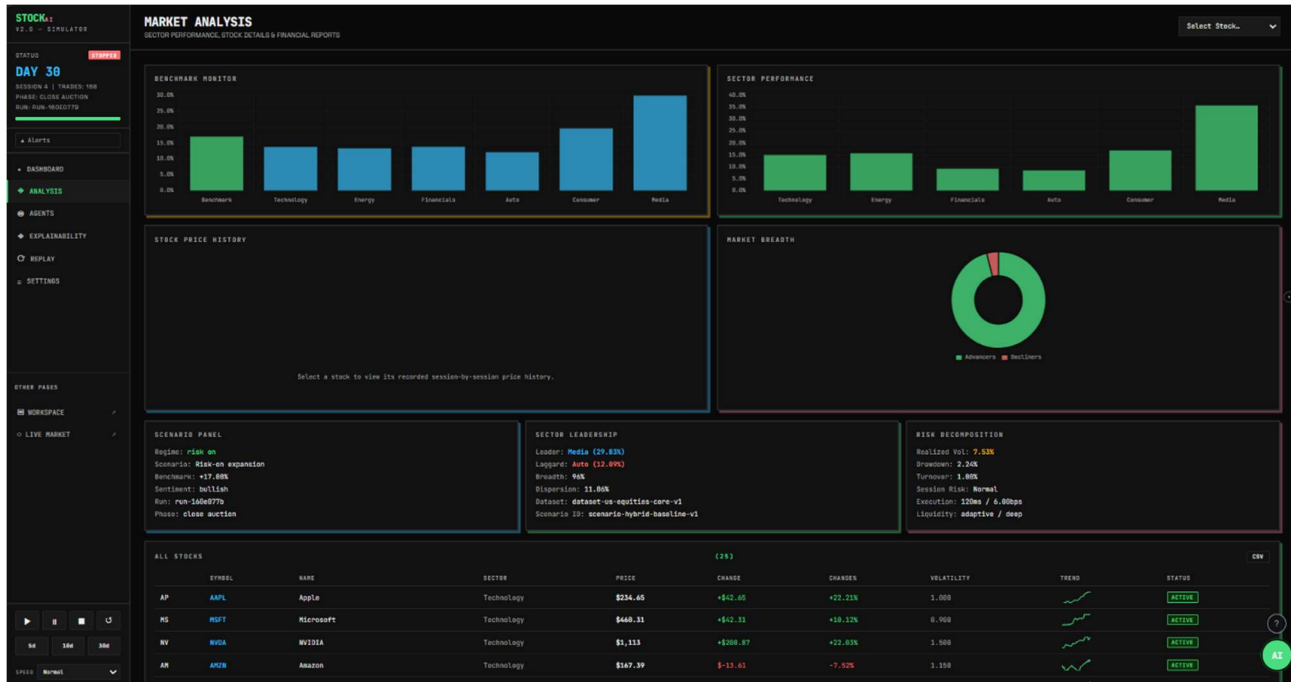


Fig 11.3: Screenshot of Simulation Dashboard

11.4 Agent Analytics and Explainability

This screen should highlight agent rankings, decision logs, consistency, conviction, and explainability information such as action distribution, bias counts, and top stocks.



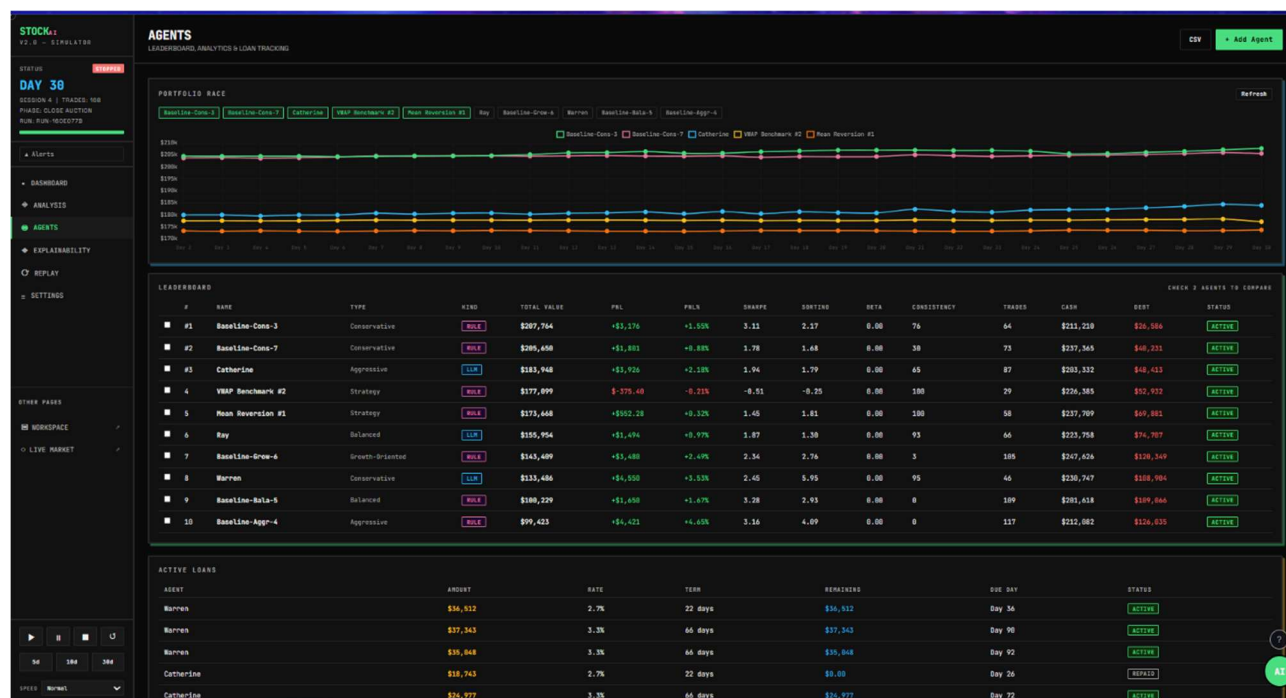


Fig 11.4: Screenshots of Agent Analytics & Explainability Panel

11.5 Live Market Intelligence

The live-market page contains: provider freshness, AI brief, market snapshot cards, simulator alignment comparison, tracked movers, sector pulse, watchlist sparkline cards, and opportunity and risk panels.

The screenshot shows the StockAI Live Performance Monitor dashboard. At the top, there's a navigation bar with the StockAI logo, a 'LIVE PERFORMANCE MONITOR' tab, and buttons for 'REFRESH TAPE', 'LANDING', 'WORKSPACE', 'SIMULATION CONSOLE', and 'LIVE MONITOR'. The main content area is divided into two columns. The left column features a large heading 'STOCKAI tracks the deployment tape.' followed by a paragraph explaining that the screen pulls in free real-world market data and frames it against StockAI's simulator regime. Below this are buttons for 'OPEN SIMULATOR' and 'JUMP TO WATCHLIST', and a status bar showing 'Live', 'Provider: Yahoo Finance', 'Freshness: Apr 18, 2:59 PM', and '8s cache age'. The right column has a heading 'BOT DEPLOYMENT BRIEF' and a sub-heading 'Risk appetite is pushing the tape higher'. It contains a paragraph about market proxies and a note about data sources. At the bottom, a 'MARKET SNAPSHOT' section has the heading 'Deployment tape, instantly readable.' and a paragraph about live proxies.

STOCKAI tracks the deployment tape.

This screen pulls in **free real-world market data**, then frames it against StockAI's simulator regime so you can compare live conditions, tracked momentum, and the bot ideas that were trained inside the research core.

[OPEN SIMULATOR](#) [JUMP TO WATCHLIST](#)

● Live Provider: Yahoo Finance Freshness: Apr 18, 2:59 PM 8s cache age

BOT DEPLOYMENT BRIEF

Risk appetite is pushing the tape higher

4/4 headline market proxies are green. 6/6 tracked equities are advancing. Consumer leads sector pulse at +2.36%. Energy is the softest pocket at -2.76%.

StockAI uses Yahoo Finance chart data without a paid key, with a 120-second backend cache to reduce polling pressure.

MARKET SNAPSHOT

Deployment tape, instantly readable.

Free live or near-live proxies let StockAI track broad market pulse, volatility pressure, and sector leadership while deployment candidates are monitored against the same backdrop.

The screenshot shows the StockAI Live Performance Monitor dashboard with the 'LIVE PERFORMANCE MONITOR' tab selected. The dashboard is divided into several sections. The top navigation bar includes the StockAI logo, 'LIVE PERFORMANCE MONITOR', and buttons for 'REFRESH TAPE', 'LANDING', 'WORKSPACE', 'SIMULATION CONSOLE', and 'LIVE MONITOR'. The main content area is split into two columns. The left column, titled 'SIMULATOR ALIGNMENT', has the heading 'Real conditions vs. the research-trained world state.' and contains three paragraphs of text. Below this is a table with three columns: 'SIMULATOR REGIME', 'BENCHMARK RETURN', and 'BREADTH'. The right column, titled 'TRACKED MOVERS', has the heading 'Where deployment pressure is building.' and contains a paragraph of text. Below this are two sections: 'ROBUSTION LEADERS' and 'PRESSURE POINTS', each with a table of tracked equities. The 'ROBUSTION LEADERS' table lists 4 tracked equities: TSLA, UNM, AAPL, and WMT. The 'PRESSURE POINTS' table lists 4 tracked equities: NFLX, XRM, AMD, and JPM.

SIMULATOR ALIGNMENT

Real conditions vs. the research-trained world state.

Real-market breadth is 0/6 green names, while the simulator is running at 96.0% breadth.

Real leadership sits in Consumer, while StockAI's simulated leader is Media.

Simulator regime is risk on, with benchmark return at +12.00%.

SIMULATOR REGIME	BENCHMARK RETURN	BREADTH
risk on	+17.88%	96.8%

TRADES	SENTIMENT	SCENARIO
168	bullish	Risk-on expansion

TRACKED MOVERS

Where deployment pressure is building.

StockAI tracks a focused equity universe to surface actionable leaders and laggards for bot monitoring without pretending to cover the entire exchange.

ROBUSTION LEADERS 4 TRACKED

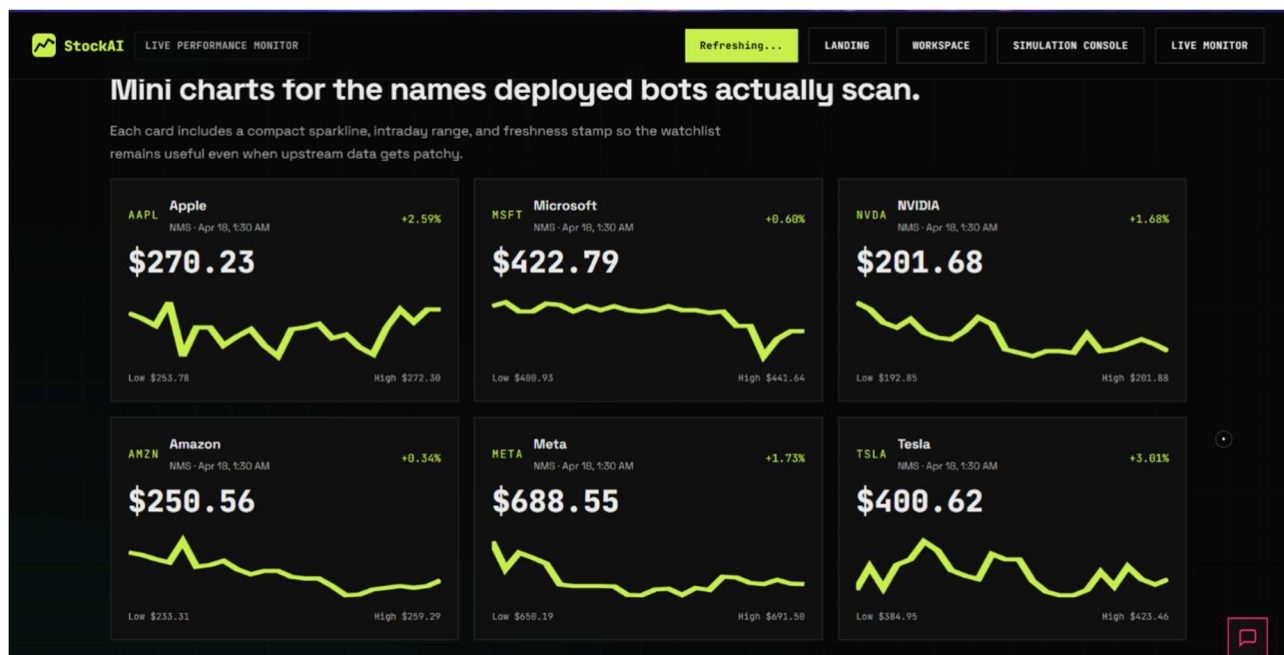
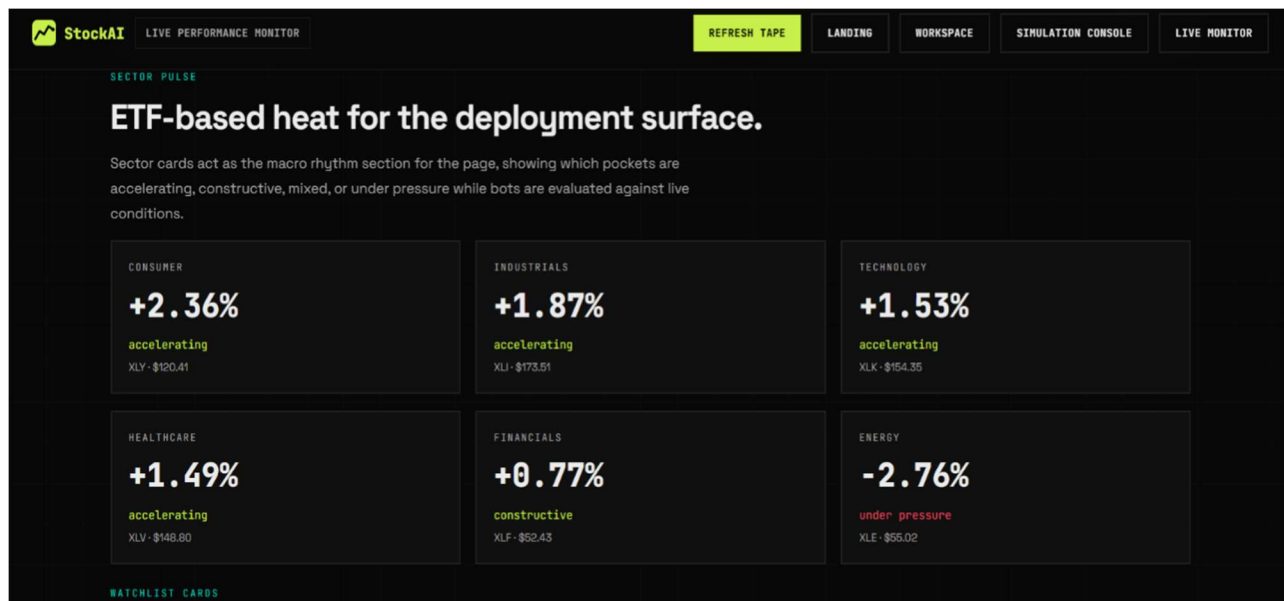
Upside Focus

TSLA	\$488.62
Tesla	+3.81%
Apr 18, 1:38 AM Featured mover	
UNM UnitedHealth	\$304.63
Apr 18, 1:38 AM	+2.68%
AAPL Apple	\$270.23
Apr 18, 1:38 AM	+2.59%
WMT Walmart	\$127.50
Apr 18, 1:38 AM	+2.18%

PRESSURE POINTS 4 TRACKED

Risk Focus

NFLX	\$97.31
Netflix	-9.72%
Apr 18, 1:38 AM Featured mover	
XRM ExactModel	\$148.44
Apr 18, 1:38 AM	-3.65%
AMD AMD	\$278.30
Apr 18, 1:38 AM	+8.85%
JPM JPMorgan	\$310.29
Apr 18, 1:38 AM	+8.11%



11.5: Screenshots of Live Market Intelligence Page

12. CONCLUSION & FUTURE WORK

12.1 Conclusion

StockAI v2.0 successfully demonstrates a substantial final-year AI and data systems project that extends beyond conventional stock-price prediction. Repository evidence confirms that the system implements a complete research-oriented simulation platform with a FastAPI backend, SQLite persistence, static browser-based interfaces, multi-agent trading logic, execution-kernel realism, and structured evaluation workflows. The project’s major achievement lies in integrating multiple domains into one coherent system: artificial intelligence through LLM-backed and rule-based agents, data systems through persistent research entities and SQLite-backed storage, financial modeling through regime-aware synthetic market behavior, and software engineering through modular APIs, tests, and deployment-ready structure. The simulation kernel includes correlated sector movement, event generation, financial reports, loans, circuit breakers, slippage, latency, and order-book matching, which significantly strengthens its realism compared with generic academic stock-analysis applications. Performance should be interpreted in terms of system capability and reproducibility rather than inflated forecasting accuracy claims. Actual repository-grounded testing confirmed successful behavior for health, stock retrieval, simulation status, workspace summary, calibration, bot creation, and evaluation endpoints. The architecture is technically sound, extensible, and academically relevant, making StockAI v2.0 a strong example of an AI-enabled financial research platform.

12.2 Future Work

Although StockAI v2.0 is already a rich platform, several important directions can improve its academic and practical value. First, classical and modern forecasting models such as ARIMA, LSTM, GRU, transformer-based predictors, and reinforcement learning agents can be integrated into the strategy framework. This would allow direct comparison between simulation-native agent behavior and trained predictive models. Second, the calibration layer can be extended from embedded priors and manual inputs to full historical ingestion pipelines using structured OHLCV datasets, macroeconomic indicators, and textual sentiment streams. This would improve realism and allow walk-forward evaluation. Third, the current live-market page can evolve into a paper-

trading or broker-linked execution layer. With suitable safeguards, future versions could connect simulation intelligence to real-time paper portfolios for closed-loop experimentation. Fourth, research analytics can be expanded through richer metrics such as turnover efficiency, execution shortfall, VaR/CVaR, sector exposure tracking, and stress-testing dashboards. Multi-user authentication, experiment versioning, and collaborative run management would also improve the platform for institutional or laboratory settings. Finally, deployment and productization can be strengthened by containerized cloud environments, CI-backed dependency management, and a more robust plugin architecture for model, strategy, and data-source extension. These improvements would transform StockAI from a strong academic prototype into a broader financial AI experimentation framework.

REFERENCES

- [1] T. M. Cover, “Universal portfolios,” *Mathematical Finance*, vol. 1, no. 1, pp. 1-29, 1991.
- [2] A. W. Lo, H. Mamaysky, and J. Wang, “Foundations of technical analysis: Computational algorithms, statistical inference, and empirical implementation,” *The Journal of Finance*, vol. 55, no. 4, pp. 1705-1765, 2000.
- [3] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Hoboken, NJ, USA: Wiley, 2015.
- [4] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735-1780, 1997.
- [5] A. Vaswani et al., “Attention is all you need,” in *Proc. Advances in Neural Information Processing Systems*, 2017, pp. 5998-6008.
- [6] J. B. Heaton, N. G. Polson, and J. H. Witte, “Deep learning for finance: Deep portfolios,” *Applied Stochastic Models in Business and Industry*, vol. 33, no. 1, pp. 3-12, 2017.
- [7] J. Moody and M. Saffell, “Learning to trade via direct reinforcement,” *IEEE Transactions on Neural Networks*, vol. 12, no. 4, pp. 875-889, 2001.
- [8] B. LeBaron, “Agent-based computational finance,” in *Handbook of Computational Economics*, vol. 2, Amsterdam, The Netherlands: Elsevier, 2006, pp. 1187-1233. DOI: [https://doi.org/10.1016/S1574-0021\(05\)02024-1](https://doi.org/10.1016/S1574-0021(05)02024-1).
- [9] D. Kahneman and A. Tversky, “Prospect theory: An analysis of decision under risk,” *Econometrica*, vol. 47, no. 2, pp. 263-291, 1979. DOI: <https://doi.org/10.2307/1914185>.
- [10] FastAPI, “FastAPI documentation,” [Online]. Available: <https://fastapi.tiangolo.com/>. [Accessed: Apr. 17, 2026].
- [11] Pydantic, “Pydantic documentation,” [Online]. Available: <https://docs.pydantic.dev/>. [Accessed: Apr. 17, 2026].
- [12] Chart.js, “Chart.js documentation,” [Online]. Available: <https://www.chartjs.org/docs/latest/>. [Accessed: Apr. 17, 2026].
- [13] Pytest, “Pytest documentation,” [Online]. Available: <https://docs.pytest.org/>. [Accessed: Apr. 17, 2026].
- [14] SQLite, “SQLite official site,” [Online]. Available: <https://www.sqlite.org/>. [Accessed: Apr. 17, 2026].
- [15] Yahoo Finance, “Finance chart endpoint used by project live-market layer,” [Online]. Available: <https://query1.finance.yahoo.com/v8/finance/chart>. [Accessed: Apr. 17, 2026].
- [16] StockAI v2.0 Repository, README.md, backend/app/, frontend/, docs/architecture.mmd,

docs/workflow.mmd, accessed from local repository analysis on Apr. 17, 2026.

[17] Groq, “Groq API documentation,” [Online]. Available: <https://console.groq.com/docs>. [Accessed: Apr. 17, 2026].

[18] OpenAI, “OpenAI API documentation,” [Online]. Available: <https://platform.openai.com/docs/>. [Accessed: Apr. 17, 2026].

[19] Google, “Google Gemini API documentation,” [Online]. Available: <https://ai.google.dev/>. [Accessed: Apr. 17, 2026].

[20] Render, “Render deployment platform,” [Online]. Available: <https://render.com/docs>. [Accessed: Apr. 17, 2026].

[21] R. Cont, “Empirical properties of asset returns: stylized facts and statistical issues,” *Quantitative Finance*, vol. 1, no. 2, pp. 223-236, 2001. DOI: <https://doi.org/10.1080/713665670>.

[22] M. D. Gould, M. A. Porter, S. Williams, M. McDonald, D. J. Fenn, and S. D. Howison, “Limit order books,” *Quantitative Finance*, vol. 13, no. 11, pp. 1709-1742, 2013. DOI: <https://doi.org/10.1080/14697688.2013.803148>.

[23] T. Fischer and C. Krauss, “Deep learning with long short-term memory networks for financial market predictions,” *European Journal of Operational Research*, vol. 270, no. 2, pp. 654-669, 2018. DOI: <https://doi.org/10.1016/j.ejor.2017.11.054>.

[24] T. L. Meng and M. Khushi, “Reinforcement Learning in Financial Markets,” *Data*, vol. 4, no. 3, 2019. DOI: <https://doi.org/10.3390/data4030110>.

[25] A. W. Lo, “The Adaptive Markets Hypothesis: Market Efficiency from an Evolutionary Perspective,” *Journal of Portfolio Management*, vol. 30, no. 5, pp. 15-29, 2004. DOI: <https://doi.org/10.3905/jpm.2004.442611>.

[26] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative Agents: Interactive Simulacra of Human Behavior,” *arXiv:2304.03442*, 2023. URL: <https://arxiv.org/abs/2304.03442>.

[27] C. Zhang, X. Liu, Z. Zhang, M. Jin, L. Li, Z. Wang, W. Hua, D. Shu, S. Zhu, X. Jin, S. Li, M. Du, and Y. Zhang, “When AI Meets Finance (StockAgent): Large Language Model-based Stock Trading in Simulated Real-world Environments,” *arXiv:2407.18957*, 2024. URL: <https://arxiv.org/abs/2407.18957>.

[28] ABIDES project, “Agent-Based Interactive Discrete Event Simulation,” GitHub repository. URL: <https://github.com/abides-sim/abides>.

APPENDIX I

RELEVANCE OF PROJECT TO COs / POs/PSOs

Title of Project	StockAI v2.0: LLM-Driven Autonomous Trading Research Platform
Implementation Details	Full-stack platform for multi-agent market simulation and strategy evaluation. Backend: Python, FastAPI, Pydantic, SQLite, Pytest. Frontend: HTML, CSS, JavaScript, Chart.js. Core Features: Order-book execution, agent-based trading, live-market tracking, research workspace, and exportable analytics.
Cost (hardware or software cost)	Minimal practical costs due to the use of free, open-source tools. Optional usage-based costs apply only if external LLM APIs are enabled. COCOMO Estimation (Commercial Equivalent): • Estimated Size: ~21.5 KLOC (Organic) • Estimated Effort: ~60 Person-Months • Assumed Rate: ₹20,000 per Person-Month • Estimated Total Cost: ₹12,00,000 (Twelve Lakh Rupees)
Type (Application, Product, Research, Review, etc.)	Research Platform / Application

Relevance		PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO 10	PO 11	PO 12	PSO1	PSO2
	CO1	3	3	2	2	3	1	1	2	2	2	2	2	3	3
	CO2	2	3	3	3	3	2	1	2	2	2	3	2	3	3
	CO3	2	2	3	2	2	1	1	1	3	2	3	2	2	2
	CO4	1	2	2	1	3	1	1	2	3	3	2	2	2	2
Justification		Applied engineering knowledge	Conducted problem analysis	Designed system solutions	Executed research	Utilized modern tools	Assessed societal impact	Considered sustainability	Applied ethical principles	Demonstrated teamwork	Communicated technical documentation	Managed project implementation	Demonstrated lifelong learning	Deployed AI solutions	Transformed data insights

Course outcomes	
CO1	Demonstrate the ability to synthesize and apply the knowledge and skills acquired in the academic program to real-world problems.
CO2	Evaluate different solutions based on economic and technical feasibility
CO3	Effectively plan a project and confidently perform all aspects of project management
CO4	Demonstrate effective written and oral communication skills

Outcome–CO Mapping	Relevance	Justification Statement for PO–PSO Mapping
PO1 - CO1	3	Applied core engineering and data science principles to build the architecture of the synthetic market simulation.
PO2 - CO1	3	Analyzed real-world financial problems to design AI agents capable of logical, adaptable trading behaviors.
PO3 - CO1	2	Designed complex system architecture, including the market kernel and real-time order-book matching mechanisms.
PO4 - CO1	2	Conducted research investigations to accurately model synthetic equities and evaluate multi-agent strategies.
PO5 - CO1	3	Utilized modern frameworks like FastAPI and the Gemini API to solve complex simulation problems.
PO6 - CO1	1	Assessed how realistic market simulations can impact broader societal understanding of financial trading.
PO7 - CO1	1	Considered sustainable, resource-efficient local computing practices over heavy, continuous cloud processing.
PO8 - CO1	2	Applied ethical principles in modeling AI agents to avoid malicious or manipulative market behaviors.
PO9 - CO1	2	Functioned effectively as a cohesive team to synthesize diverse academic knowledge into a single platform.
PO10 - CO1	2	Documented the practical application of AI concepts in clear, real-world technical reports.
PO11 - CO1	2	Managed the implementation of complex engineering knowledge within strict 12-week project deadlines.
PO12 - CO1	2	Demonstrated continuous learning by applying cutting-edge LLM research directly into the simulation engine.

PSO1 - CO1	3	Deployed scalable, ethical AI solutions to simulate dynamic stock market environments effectively.
PSO2 - CO1	3	Transformed complex order-book data streams into actionable insights for the trading simulation dashboard.
PO1 - CO2	2	Evaluated open-source engineering tools to ensure the system architecture is technically and economically viable.
PO2 - CO2	3	Analyzed various backend frameworks to solve the latency problems of handling live-market data.
PO3 - CO2	3	Designed system solutions that strictly prioritized a zero-cost baseline while maintaining high performance.
PO4 - CO2	3	Investigated different data management solutions (SQLite vs. heavy cloud DBs) for maximum economic feasibility.
PO5 - CO2	3	Selected cost-effective modern tools like Chart.js over expensive, proprietary dashboard software.
PO6 - CO2	2	Assessed the societal benefit of creating an accessible, low-cost financial research platform for academic use.
PO7 - CO2	1	Ensured environmental sustainability by evaluating and designing a lightweight, low-compute architecture.
PO8 - CO2	2	Maintained professional ethics by accurately evaluating and reporting hardware and software cost models.
PO9 - CO2	2	Collaborated as a team to reach consensus on the most technically feasible and economically sound project stack.
PO10 - CO2	2	Communicated the economic and technical platform evaluations clearly within the project documentation.
PO11 - CO2	3	Applied specific cost-estimation models (COCOMO) to manage project feasibility and budget planning.
PO12 - CO2	2	Engaged in self-learning to continuously evaluate the latest, most cost-effective AI integration methods.
PSO1 - CO2	3	Ensured the designed AI models and solutions were both highly scalable and economically practical to run.
PSO2 - CO2	3	Evaluated the most efficient data transformation tools to minimize processing costs and overhead.

PO1 - CO3	2	Applied systematic engineering principles to define the project scope, roadmap, and lifecycle phases.
PO2 - CO3	2	Analyzed potential scheduling bottlenecks and complex task dependencies during the initial planning phase.
PO3 - CO3	3	Designed a structured development roadmap to ensure the orderly integration of the backend and frontend systems.
PO4 - CO3	2	Researched various project management methodologies to seamlessly execute the 12-week development plan.
PO5 - CO3	2	Utilized modern project management visualization and Gantt charting tools for accurate timeline tracking.
PO6 - CO3	1	Planned project deliverables carefully to meet the required academic and societal safety standards.
PO7 - CO3	1	Planned sustainable, structured work schedules to ensure continuous, burnout-free module development.
PO8 - CO3	1	Committed to professional academic ethics by strictly meeting planned milestones and review deadlines.
PO9 - CO3	3	Divided complex development modules among team members to demonstrate effective collaborative planning.
PO10 - CO3	2	Communicated the project timeline and management strategies visually through detailed Gantt charts.
PO11 - CO3	3	Confidently performed all aspects of project management, from initial problem definition to final delivery.
PO12 - CO3	2	Learned and adapted advanced time-management techniques as the simulation's complexities evolved.
PSO1 - CO3	2	Planned the systematic, phased deployment of the AI-driven market simulation platform to ensure stability.
PSO2 - CO3	2	Managed the strict timeline required for developing and testing the data transformation and analytics pipelines.
PO1 - CO4	1	Communicated core engineering architectures clearly and professionally through formal academic writing.
PO2 - CO4	2	Articulated problem analysis and logical agent workflows accurately in detailed technical documents.

PO3 - CO4	2	Created comprehensive UML and system design diagrams to visually explain the platform's architecture.
PO4 - CO4	1	Documented the research findings, evaluation workflows, and experiment metrics effectively for peer review.
PO5 - CO4	3	Utilized modern documentation and presentation tools to showcase the final project deliverables.
PO6 - CO4	1	Clearly communicated the platform's societal relevance and academic purpose in the abstract and introduction.
PO7 - CO4	1	Documented the sustainable, lightweight design choices thoroughly in the final project report.
PO8 - CO4	2	Ensured all written reports and oral presentations adhered strictly to academic integrity and ethical guidelines.
PO9 - CO4	3	Collaborated seamlessly as a team to co-author comprehensive reports and deliver group presentations.
PO10 - CO4	3	Demonstrated mastery in technical writing, complex report formatting, and oral project defense.
PO11 - CO4	2	Documented project management milestones, cost estimates, and execution phases clearly for evaluators.
PO12 - CO4	2	Improved communication skills continuously through iterative review feedback and documentation updates.
PSO1 - CO4	2	Communicated the design, logic, and deployment strategies of the AI trading agents with high clarity.
PSO2 - CO4	2	Presented complex data analytics and trading insights through accessible, well-designed visual charts.

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO1:

Design and deploy scalable, ethical AI and data-driven solutions using AI, ML, and Data Science across domains.

PSO2:

Analyse and transform complex data into actionable insights and reliable decision-making using modern tools and algorithms.

PROGRAM OUTCOMES (PO's)

I. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

II. Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

III. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

IV. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

V. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

VI. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

VII. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

VIII. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

IX. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

X. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

XI. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

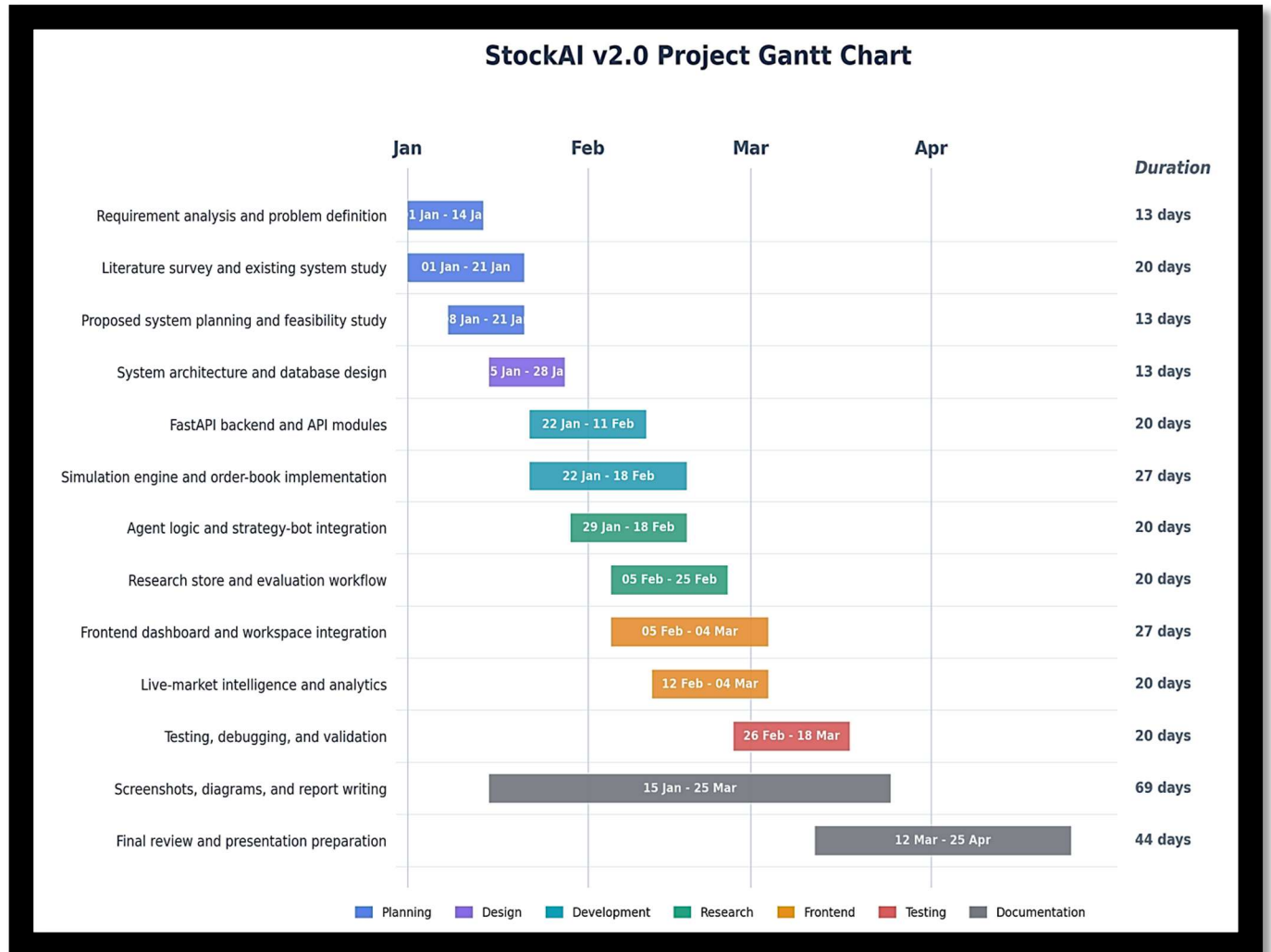
XII. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

APPENDIX II

PROJECT ACTIVITIES DURATION CHART

Activity	Plan start (No. of weeks)	Plan duration (No. of weeks)	Actual start (No. of weeks)	Actual Duration (No. of weeks)	Percent complete
Requirement Analysis & Problem Definition	1	2	1	2	100%
Literature Survey & Feasibility Study	2	1	2	1	100%
System Architecture & Database Design	2	2	2	3	100%
Backend API & Simulation Engine	3	2	3	2	100%
Agent Logic & Bot Integration	5	1	5	1	100%
Frontend Dashboard & Analytics	5	2	5	2	100%
Testing, Debugging & Validation	6	4	6	5	100%
Report Writing & Documentation	10	2	11	2	100%
Final Review & Presentation Prep	4	8	5	8	100%
Requirement Analysis & Problem Definition	12	1	12	1	100%

GANTT CHART



Gantt Chart Explanation

The Gantt chart illustrates the development schedule of StockAI v2.0, spanning from January 2026 to April 2026 across 13 activities and 7 phases. The project begins with Planning (requirement analysis, literature survey, feasibility study) in January, followed by System Design and core Software Development (backend, simulation engine, agent integration) from late January through February. Frontend development, Research workflow, and Testing activities run concurrently from February to mid-March, reflecting the Agile methodology adopted for the project. Documentation runs in parallel throughout the entire project (69 days), while Final review and presentation preparation extend until late April (44 days). The critical path flows from Planning, Design, Development, Testing, Final Review, with a total project duration of approximately 115 days.

APPENDIX III

Project Setup and Execution

```
# Step 1: Create virtual environment
python -m venv .venv

# Step 2: Activate virtual environment on Windows
.venv\Scripts\activate

# Step 3: Install dependencies
pip install -r requirements.txt

# Step 4: Run the application
python backend/run.py
```

Running Tests

```
pytest tests -q
```

Application URLs

```
Landing Page:
http://127.0.0.1:8000/

Research Workspace:
http://127.0.0.1:8000/workspace

Simulation Dashboard:
http://127.0.0.1:8000/app

Live Market Intelligence:
http://127.0.0.1:8000/live-market

API Health Check:
http://127.0.0.1:8000/health
```

Important API Endpoints

```
GET /health
Returns backend health status.

GET /market/stocks
Returns all available stocks and current prices.

GET /simulation/status
Returns current simulation day, session, trades, agents, and analytics.

POST /simulation/start
Starts the simulation.

POST /simulation/pause
Pauses the simulation.

POST /simulation/reset
Resets the simulation state.

GET /agents
Returns all active simulation agents.
```

```
GET /agents/{agent_id}/analytics
Returns performance and risk metrics for a selected agent.

GET /api/live-market/snapshot
Returns live-market intelligence data with fallback support.

GET /runs
Returns research simulation runs.

GET /datasets
Returns available research datasets.
```